

Would I Instagram This? From Pixels to Plain English

CS156 – Machine Learning Pipeline Assignment 3

1 What's New

Assignments 1 and 2 established that my Instagram sharing decisions are predictable from visual features, but the best models remained opaque - I could not explain any individual prediction in words. This assignment introduces two novel techniques chosen specifically to close that interpretability gap, and to test whether the dominant signal is visual texture or semantic scene content.

Component	Status	Why it's here
Handcrafted LR	Reused (Assignment 1)	Low-level baseline: explicit pixel statistics. Establishes the floor.
CNN + SVM (RBF)	Reused (Assignment 2)	Best purely visual model (81.7% test). Ceiling for texture.
Caption LR (BLIP → MiniLM)	NEW (Assignment 3)	Tests whether English descriptions predict sharing. Interpretable.
Fine-tuned ViT + Attention Rollout	NEW (Assignment 3)	Tests global self-attention and provides spatial attention maps.

The two reused models are controlled baselines - any performance difference in Assignment 3 is attributable to the new representations, not a different split. The same `train_test_split(test_size=0.2, random_state=42, stratify=y)` is used across all three assignments.

2 Introduction and Research Question

2.1 The Journey So Far

- Assignment 1.** The starting question was whether low-level pixel statistics could predict sharing behaviour. Logistic regression on 13 handcrafted features achieved **78.3% test accuracy, AUC 0.837**. The dominant signal was brightness: shared photos had `mean_brightness` of 96.0 vs. 124.8 for personal (Cohen's $d = -0.976$), and `brightness_var` of 2615 vs. 3911 ($d = -1.090$). The model had effectively learned the signature of deliberate Lightroom editing. Naive Bayes reached only 65.0% because feature independence is violated (`colorfulness↔saturation` $r = 0.73$, `face↔face_size` $r = 0.81$). The critical limitation: every feature described *how* the photo looked but nothing about *what* it depicted.
- Assignment 1. CLIP extension.** Frozen CLIP embeddings with logistic regression jumped to **88.3% test accuracy, AUC 0.946**, a 10 percentage point gain. This was the first clear signal that semantic content mattered. But CLIP is a 151M-parameter black box with no human-readable interpretation of its embedding dimensions.
- Assignment 2.** Tested three further representations (frozen CNN ResNet-18, fine-tuned CNN, CLIP zero-shot) and two additional classifiers (SVM RBF, Random Forest). CNN + SVM reached **81.7% (AUC 0.893)**. Fine-tuned ResNet-18 reached 83.3% but showed clear overfitting. CLIP + LR remained the best at 88.3%. The winning model still offered no explanation for its predictions.
- Assignment 3. The gap Assignment 3 closes.** Can an *interpretable* semantic pipeline approach CLIP's performance using automatically generated text descriptions? And can a Vision Transformer, which unlike a CNN sees the whole image at once, provide spatial attention maps that reveal the compositional elements driving the decision?

Table 1: Prior results carried forward from Assignments 1 and 2.

Assignment	Model	Features	Test Acc.	AUC
Assignment 1	Handcrafted LR	13 pixel stats	0.783	0.837
Assignment 1	Naive Bayes	13 pixel stats	0.650	0.776
Assignment 1	CLIP + LR	512d CLIP	0.883	0.946
Assignment 2	CNN + SVM (RBF)	512d ResNet-18	0.817	0.893
Assignment 2	Fine-tuned CNN	Raw pixels	0.833	0.870
Assignment 2	CLIP Zero-Shot	Text prompts	0.733	0.878

2.2 The Central Research Question

Is my Instagram sharing behaviour driven more by *how* a photo looks, its pixel statistics and visual texture, or by *what* it depicts, its semantic scene content and compositional structure?

Prior assignments optimised for accuracy. This one optimises for *understanding* based on the feedback. CLIP already showed that semantic alignment predicts sharing better than pixel statistics, the question now is whether that semantic signal can be made legible. If automatically generated captions predict sharing, the content of the description *is* the explanation. If the ViT’s attention maps concentrate on compositional anchors, those maps *are* the explanation.

2.3 Hypotheses

H1 - The Caption Hypothesis

When I think about what makes a photo postable for me, I can almost always describe it in a sentence: the archways in Taiwan’s Pagodas framing a sunset; the fog sitting low over the harbour at dusk; a corner of a café with one beam of light on an empty table. My personal photos resist that kind of description: a bright screenshot, a quick food shot before eating, a group photo in ambient light. If I read those captions without seeing the photos, I think I could mostly guess which ones I’d post. The idea is that the words alone should be enough. Based on this reasoning:

H1: A logistic regression classifier trained exclusively on sentence embeddings of BLIP-generated captions will achieve test accuracy $\geq 78.3\%$ (the handcrafted LR floor) and $AUC \geq 0.85$. The more meaningful benchmark is CLIP + LR at 88.3%, so if Caption LR approaches this, it confirms that objective text descriptions capture approximately the same semantic signal as a 151M-parameter vision-language model.

Falsification: If Caption LR $< 78.3\%$, BLIP-generated semantic content does not predict sharing at even the level of explicit pixel statistics, either the aesthetic is more visual than semantic, or BLIP’s descriptions miss the specific properties that matter (mood, editing style, intentionality).

H2 - The ViT Attention Hypothesis

Looking back at the Grad-CAM maps from Assignment 2, what struck me was how often the activation concentrated on structural frames: the arch itself, the window edge, the horizon line where sky meets water. My posting instinct is about composition: whether there’s something for the eye to move toward, whether the frame feels deliberate. CNNs build up local textures into global features gradually. A ViT processes the whole image as a set of patches where every patch can directly attend to every other from layer one. If composition is what distinguishes my shared photos, a model that sees the whole image simultaneously should capture it more directly.

H2(a): A fine-tuned Vision Transformer will achieve test accuracy $\geq 88.3\%$ (matching CLIP + LR). If the ViT outcome is between 81.7% and 88.3%, it would mean global attention provides some compositional signal not captured by local CNN texture, but not enough to match frozen semantic embeddings.

H2(b): If global self-attention captures compositional structure, then attention rollout maps for correctly classified shared photos will show concentrated activation on structural compositional elements (architectural frames, horizon lines, and foreground-background separation) rather than on faces, isolated objects, or diffuse regions.

Falsification: If attention maps are diffuse even for correctly classified shared photos, the model may be relying on shortcut features (overall brightness, colour distribution) rather than compositional structure.

3 Data

3.1 Dataset Overview

The dataset consists of 300 personal photos drawn from my own digital archive, evenly split between 150 shared ($y = 1$) and 150 personal ($y = 0$), self-annotated before any feature extraction, and perfectly balanced so that accuracy is a valid metric without resampling.

Sample Photos: Shared (top) vs Personal (bottom)



Figure 1: Sample photos. Top row: 6 shared. Bottom row: 6 personal. Shared photos tend toward darker exposures and deliberate compositions (architectural details, waterfalls, scenic interiors). Personal photos are brighter, more candid (group selfies, passports, close-up portraits, food markets). The distinction is not subject matter alone but *intentionality and editing*.

3.2 Labelling and Its Implications

Labels reflect aesthetic preferences at a specific moment. Annotation happened before any modelling, so there is no label leakage. Two sources of noise set an irreducible accuracy ceiling, estimated in §9.4:

1. Borderline cases that could have been labelled either way on a different day.
2. Decisions made for non-visual reasons: sentimental value, the caption I planned to write, a specific context invisible to any model.

3.3 Sampling

The photos were sampled to deliberately include ambiguous cases across varying lighting conditions, locations, and subjects, ensuring the task reflects genuine decision-making rather than an artificially clean separation (see Assignment 1 for the full sampling rationale).

4 Preprocessing and Feature Extraction

4.1 Common Preprocessing

`StandardScaler` is fit on the training set only, then applied to test. Critical for LR and SVM because `sharpness` runs in the thousands while `edge_density` sits in $[0, 1]$. Neural models (ViT) use their own internal ImageNet normalisation.

4.2 Handcrafted Features (Reused, Appendix A-2)

13 features extracted in Assignment 1: `mean_brightness`, `brightness_var`, `mean_saturation`, `saturation_var`, `warmth`, `colorfulness`, `mean_hue`, `hue_var`, `sharpness`, `edge_density`, `center_weight`, `face_present`, `face_size_ratio`. See Assignment 1 for full definitions.

4.3 CNN Features - Frozen ResNet-18 (Reused, Appendix A-3)

512-dimensional penultimate-layer activations from frozen pretrained ResNet-18 with ImageNet normalisation and 224×224 centre crop. Reused from Assignment 2 without modification.

4.4 BLIP Caption Generation (NEW, Appendix A-4)

What BLIP is and why it is used here.

BLIP (Bootstrapping Language-Image Pre-training, Li et al. 2022) is a vision-language model trained on 129M image-text pairs using three simultaneous objectives: image-text contrastive learning (ITC), image-text matching (ITM), and image-conditioned language modelling (LM). For caption generation, the visual encoder (a frozen ViT-B/16) produces patch embeddings $\{\mathbf{v}_1, \dots, \mathbf{v}_N\} \in \mathbb{R}^d$. These are fed as cross-attention context into an autoregressive language decoder, which generates captions token by token:

$$P(w_1, \dots, w_T \mid \text{image}) = \prod_{t=1}^T P(w_t \mid w_{<t}, \{\mathbf{v}_1, \dots, \mathbf{v}_N\}) \quad (1)$$

The decoder attends over all visual patch embeddings at every generation step. Formally, each token is drawn from

$$P(w_t \mid w_{<t}, \text{image}) = \text{softmax}(\mathbf{W}_o \cdot \text{CrossAttn}(\mathbf{Q} = \mathbf{h}_{t-1}, \mathbf{K} = \mathbf{V} = \mathbf{V})) \quad (2)$$

where $\mathbf{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_N\}$ are the visual encoder’s patch embeddings. This is what allows BLIP to produce descriptions grounded in specific image regions rather than generic boilerplate.

Why not CLIP for this purpose?

CLIP produces embeddings but cannot generate text. BLIP generates captions automatically from the image content, removing the need for user-written prompts. The test of H1 is specifically: can an *objective, automatically generated description* predict sharing? Using CLIP zero-shot would re-introduce my own linguistic assumptions - exactly what the hypothesis is designed to avoid. **Implementation.**

Salesforce/blip-image-captioning-base via HuggingFace. Generation uses beam search with `num_beams=3`, `max_length=50`, `repetition_penalty=1.8`, and `no_repeat_ngram_size=2`. The repetition penalty was added after an initial run produced degenerate outputs on a handful of images (*e.g.* “yo yo yo”, “self self self”, “chiang chiang chiang”); the new parameters largely resolved these, though the bottom-5 P(shared) list (§9.1.3) still shows one residual artefact.

4.5 Sentence Embedding - MiniLM (NEW, Appendix A-5)

Captions are strings; the classifier needs vectors. `all-MiniLM-L6-v2` (Reimers & Gurevych, 2019) encodes each caption as a 384-dimensional dense vector. The model is a 6-layer transformer trained via contrastive learning on 1B sentence pairs. For caption c :

$$\mathbf{h} = \text{MeanPool}(\text{TransformerEncoder}(c)) \in \mathbb{R}^{384} \quad (3)$$

Semantic similarity is

$$\text{sim}(c_1, c_2) = \frac{\mathbf{h}_1 \cdot \mathbf{h}_2}{\|\mathbf{h}_1\| \|\mathbf{h}_2\|} \quad (4)$$

So “a dark archway framing a sunset” and “an architectural composition at dusk” will have high cosine similarity even though they share no words, the embedding captures meaning, not surface form. This is preferable to TF-IDF on captions of 8–15 words, which would produce extremely sparse representations and miss semantic equivalences entirely. Output: (300, 384) matrix. This is the feature matrix for Caption LR.

4.6 ViT Data Preparation (NEW, Appendix A-6)

Raw pixels resized to 224×224, normalised with ImageNet mean/std. Training augmentation: `RandomResizedCrop(224, scale=(0.8, 1.0))`, `RandomHorizontalFlip(p=0.5)`, `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)`. Applied to training set only; validation and test use clean transforms. With only 192 training images, the ViT must not memorise specific pixel patterns; augmentation forces invariance to minor photographic variations and slows overfitting.

4.7 Exploratory Data Analysis (Appendix A-7)

4.7.1 Cohen’s d (Handcrafted Features)

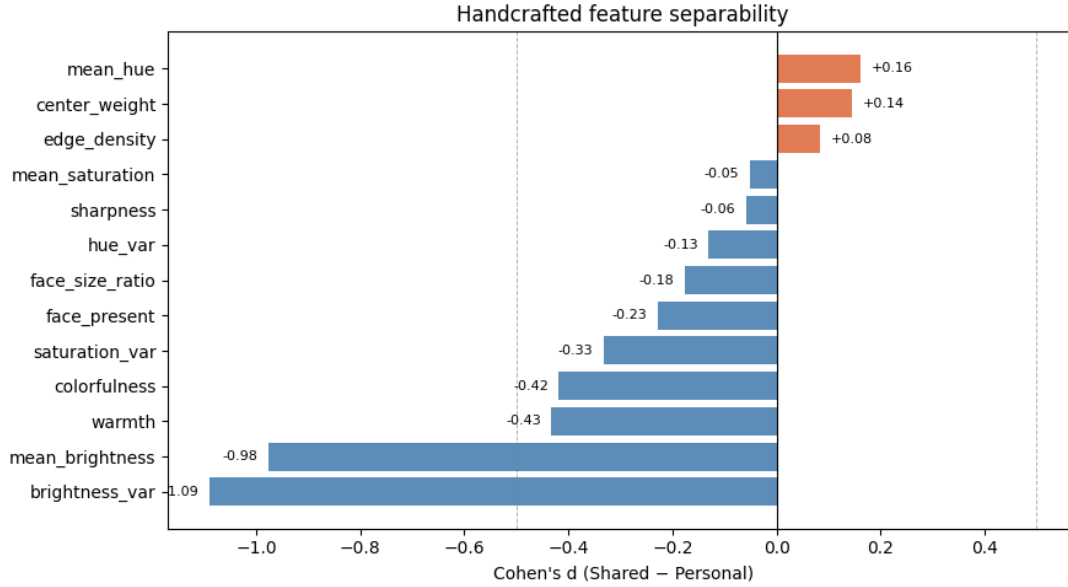


Figure 2: Cohen’s d per handcrafted feature. Orange = higher in Shared; Blue = higher in Personal. Only **brightness_var** ($d = -1.090$) and **mean_brightness** ($d = -0.976$) cross the large-effect threshold. Dashed lines at $d = \pm 0.5$ mark the medium-effect boundary. Even the two strongest handcrafted features only describe *how bright* and *how variable* a photo is - they say nothing about what it depicts.

4.7.2 Caption Word Frequency (NEW)

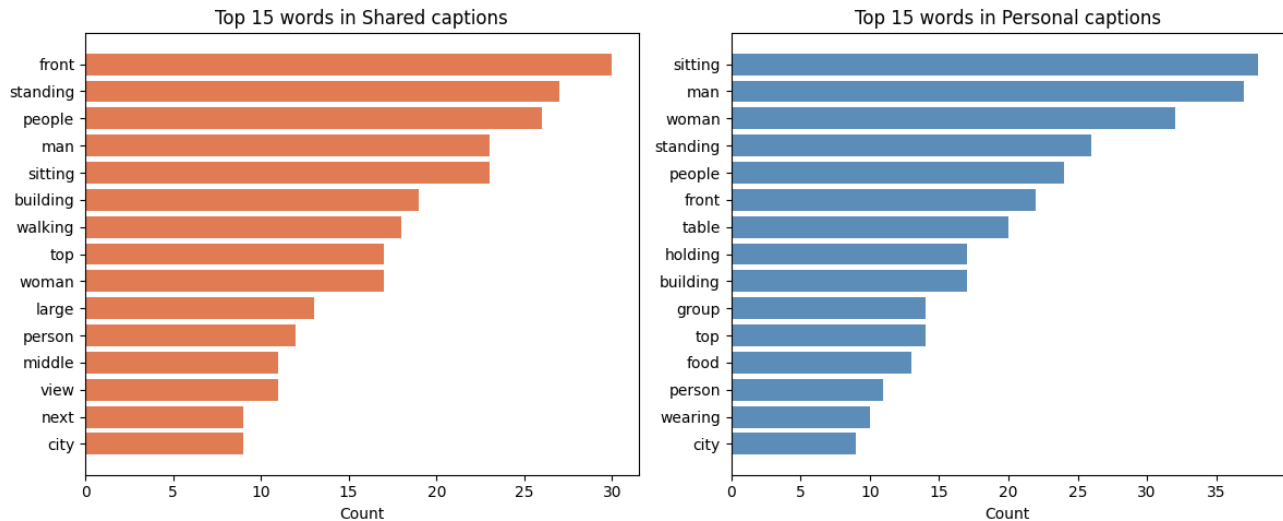


Figure 3: Top-15 words in shared (left, orange) vs. personal (right, blue) captions after stopword removal. Shared-only words: *large, middle, next, view, walking* - spatial and scenic descriptors. Personal-only words: *food, group, holding, table, wearing* - objects and activities. Overlap (noise for the classifier): *building, city, front, man, people, person, sitting, standing, top, woman*.

The overlap is substantial (10 of 15 words appear in both lists). This is because BLIP’s vocabulary is generic - both shared and personal photos contain buildings, people, and common spatial descriptors. The separation that *does* exist is qualitatively correct (scenic for shared, domestic for personal), but it must be extracted from the *combination* of words - which is precisely what MiniLM embeddings encode and what LR can exploit. This previews why Caption LR achieves only 75.0% accuracy: the semantic signal exists but is subtle.

4.7.3 Embedding Space - PCA and t-SNE



Figure 4: Left: PCA 2D on 13-dim handcrafted features ($PC1 \approx 18.5\%$, $PC2 \approx 17.2\%$ variance). Right: t-SNE 2D on 384-dim caption embeddings. Both show substantial overlap between classes, but the geometry differs - the handcrafted PCA is a near-uniform cloud, while the caption t-SNE shows some directional structure. Neither produces clean clusters, which previews the quantitative result: caption embeddings do not give Caption LR a decisive advantage over handcrafted LR.

4.7.4 Class-Conditional Statistics

Table 2: Class-conditional means. Shared photos are consistently darker and lower-contrast, consistent with deliberate Lightroom editing. Face presence is nearly twice as common in personal photos.

Feature	Personal ($y = 0$)	Shared ($y = 1$)
mean_brightness	124.8	96.0
brightness_var	3910.7	2615.5
face_present	0.15	0.08

5 Analysis Design and Train/Test Split

5.1 Task Definition

Binary classification: given feature vector $\mathbf{x}$, predict $y \in \{0, 1\}$ where 1 = shared and 0 = personal. The core design is a **controlled experiment**: where possible, hold the classifier constant (logistic regression) and vary only the feature representation. This isolates the contribution of each representation type. The one exception is CNN features, where SVM RBF is used because Assignment 2 empirically established that LR on CNN features (80.0%) substantially underperforms SVM RBF (81.7%) - the class boundary in CNN embedding space is nonlinear.

5.2 Train/Test Split (Appendix A-8)

Identical to Assignment 1 and Assignment 2: `train_test_split(test_size=0.2, random_state=42, stratify=y)`.

- Training: 240 images (120 shared, 120 personal)
- Test: 60 images (30 shared, 30 personal)

Using the *identical* split means any performance difference between Assignment 3 models and prior models is attributable to the representation, not a different data partition. Stratification guarantees exact 50/50 class proportions in every fold: without it, fold variance would be $\sigma = \sqrt{n_k \cdot \pi(1 - \pi)} = \sqrt{48 \cdot 0.25} \approx 3.5$ images per fold.

5.3 Validation Strategy

5-fold stratified CV on the training set (240 images). Each fold: 192 train, 48 validation.

$$\widehat{\text{Acc}}_{\text{CV}} = \frac{1}{K} \sum_{k=1}^K \text{Acc}_k \quad (5)$$

More reliable than a single 10% holdout (24 images – too noisy). ViT fine-tuning uses a single 80/20 split within training (192 train, 48 val) with early stopping - consistent with Assignment 2's neural-model protocol.

6 Model Selection and Mathematical Foundations

Four models, each chosen to answer a specific part of the research question. Handcrafted LR = what pixel statistics alone can tell us. CNN + SVM = what learned visual textures tell us. Caption LR = what objective text descriptions tell us. Fine-tuned ViT = what global compositional attention tells us.

6.1 Logistic Regression on Handcrafted Features (Appendix A-9)

Establishes the interpretable pixel-stats floor (78.3% test accuracy from Assignment 1). Any model failing to exceed this has not extracted meaningful signal beyond raw pixel properties. Full derivation in Assignment 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \quad (6)$$

$$L(\mathbf{w}) = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \frac{1}{2C} \|\mathbf{w}\|^2 \quad (7)$$

Config: `penalty='l2', C=1.0, solver='lbfgs', max_iter=1000`.

6.2 SVM with RBF Kernel on CNN Features (Appendix A-10)

Assignment 2 showed that LR on CNN features (80.0%) substantially underperforms SVM RBF (81.7%). The gap confirmed that the class boundary in ResNet-18 embedding space is nonlinear - the RBF kernel is required to exploit it. This model represents the ceiling of purely texture-based visual approaches without semantic content. Full derivation in Assignment 2.

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (8)$$

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (9)$$

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right) \quad (10)$$

Hyperparameters from Assignment 2 grid search (rerun in Assignment 3): $C = 10.0$, $\gamma = \text{'scale'}$.

6.3 Caption LR - BLIP → MiniLM → Logistic Regression (NEW, Appendix A-11)

Why this pipeline tests H1.

Caption LR is the only model that makes its prediction based entirely on a text description of the photo - no pixel values, no learned visual features. If it predicts sharing well, the *content* of what BLIP describes is sufficient. Logistic regression is chosen over SVM or RF specifically for interpretability: predictions can be connected back to the caption language by inspecting which captions receive the highest $P(\text{shared})$.

Why LR and not SVM on caption embeddings.

SVM could achieve higher accuracy but would obscure the semantic signal. The goal for this model is understanding, not maximising accuracy.

LR formulation.

Identical to §5.1, but $\mathbf{x} \in \mathbb{R}^{384}$ rather than $\mathbb{R}^{13}$. With 384 dimensions and 240 training samples (underdetermined

regime), strong regularisation is expected - grid search over $C \in \{0.001, 0.01, 0.1, 1.0, 10.0\}$, with smaller C expected to win (consistent with Assignment 1/Assignment 2 CLIP + LR at $C = 0.001$).

Pseudocode:

```

Input: 300 images

Phase 1 - Caption Generation (A-4):
  for each image i:
    caption_i = BLIP.generate(image_i, beam_search, rep_penalty=1.8)

Phase 2 - Sentence Embedding (A-5):
  for each caption_i:
    embedding_i = MiniLM.encode(caption_i)          # (384,)
  X_caption = stack(embedding_1, ..., embedding_300) # (300, 384)

Phase 3 - Classification (A-11):
  Split X_caption -> X_train_cap (240x384), X_test_cap (60x384)
  StandardScaler(fit on train only)
  for C in [0.001, 0.01, 0.1, 1.0, 10.0]:
    CV_acc[C] = 5-fold stratified CV accuracy of LR(C)
  best_C = argmax(CV_acc)
  LR(best_C).fit(X_train_cap, y_train)
  Predict on X_test_cap -> accuracy, F1, AUC

```

6.4 Fine-tuned ViT + Attention Rollout (NEW, Appendix A-12)

Why ViT and not more ResNet fine-tuning.

Assignment 2 fine-tuned ResNet-18 achieved 83.3% test accuracy - below CLIP + LR (88.3%). The primary reason was overfitting: 8.4M trainable parameters on 192 training images. ViT fine-tuning faces the same risk, and H2(a) explicitly acknowledges this. The justification for ViT is not primarily accuracy - it is the **attention mechanism**. ResNet Grad-CAM is a post-hoc gradient approximation applied *after* the forward pass. ViT self-attention *is* the model's actual computation. Attention rollout traces information flow through the model's own mechanism, not an external proxy.

Why global self-attention matters for composition.

ResNet-18 builds its final representation through many steps of local aggregation - early layers have small receptive fields (3×3), and only the final layers see the whole image indirectly. A ViT processes the image as a set of $N = 196$ patches (16×16 pixels each) where every patch can attend to every other from layer 1. Long-range spatial relationships - a foreground arch relating to a background horizon, a window frame containing a landscape - are modelled explicitly rather than implicitly assembled from local parts. This is the architectural property that makes ViT the right tool for testing H2(b).

6.4.1 ViT Architecture

Input $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is split into $N = HW/P^2$ patches of size $P \times P$. Each patch is linearly projected:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{E}\mathbf{x}_p^1; \dots; \mathbf{E}\mathbf{x}_p^N] + \mathbf{E}_{\text{pos}} \quad (11)$$

where $\mathbf{E} \in \mathbb{R}^{(P^2 C) \times d}$ is the patch embedding matrix, $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times d}$ is the positional embedding, and $\mathbf{x}_{\text{class}}$ is a learnable classification token. For vit-base-patch16-224: $P = 16$, $N = 196$, $d = 768$, $L = 12$ layers, 12 heads.

Each layer ℓ applies multi-head self-attention followed by a feedforward MLP:

$$\mathbf{Q}^h = \mathbf{z}^{\ell-1} \mathbf{W}_Q^h, \quad \mathbf{K}^h = \mathbf{z}^{\ell-1} \mathbf{W}_K^h, \quad \mathbf{V}^h = \mathbf{z}^{\ell-1} \mathbf{W}_V^h \quad (12)$$

$$\mathbf{A}^h = \text{softmax} \left(\frac{\mathbf{Q}^h (\mathbf{K}^h)^\top}{\sqrt{d/n_{\text{heads}}}} \right) \in \mathbb{R}^{(N+1) \times (N+1)} \quad (13)$$

$$\mathbf{z}'^\ell = \text{concat}(\text{head}^1, \dots, \text{head}^{n_{\text{heads}}}) \mathbf{W}_O + \mathbf{z}^{\ell-1} \quad [\text{residual}] \quad (14)$$

$$\mathbf{z}^\ell = \text{MLP}(\text{LayerNorm}(\mathbf{z}'^\ell)) + \mathbf{z}^\ell \quad (15)$$

Classification uses the final-layer [CLS] token: $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{W}_{\text{head}} \cdot \mathbf{z}_{\text{class}}^L + b_{\text{head}})$.

6.4.2 Attention Rollout (Abnar and Zuidema, 2020)

The problem with raw attention. In a 12-layer transformer, a patch’s influence on [CLS] passes through residual connections *as well as* attention. A single layer’s attention weights can be misleading - a patch with low attention at layer 3 may still influence the final prediction via the residual shortcut.

The solution. At each layer, account for both attention and residual by computing an effective attention matrix:

$$\hat{\mathbf{A}}^\ell = 0.5 \cdot \bar{\mathbf{A}}^\ell + 0.5 \cdot \mathbf{I} \quad (16)$$

where $\bar{\mathbf{A}}^\ell$ is the mean attention across the 12 heads at layer ℓ , and $\mathbf{I}$ represents the residual. The rollout matrix is the cumulative product across all layers:

$$\mathbf{R} = \hat{\mathbf{A}}^1 \cdot \hat{\mathbf{A}}^2 \dots \hat{\mathbf{A}}^L \in \mathbb{R}^{(N+1) \times (N+1)} \quad (17)$$

The first row, $\mathbf{R}[0, 1 :]$, gives the total attention each of the $N = 196$ spatial patches receives from [CLS] after propagation through all 12 layers and all residual connections. Reshape to 14×14 , upsample to 224×224 , overlay on the original image. Patches with high values in $\mathbf{R}[0, 1 :]$ contributed most to the final decision through the model’s own computational mechanism, not a post-hoc approximation.

Pseudocode:

```
Input: image x, fine-tuned ViT (L=12 layers)

Forward pass - save attention weights:
for each layer L in 1..12:
    A_raw[L] = attention_weights(z^{L-1})    # (n_heads, N+1, N+1)
    A_mean[L] = mean(A_raw[L], axis=0)      # (N+1, N+1)

Rollout:
R = Identity(N+1)
for each layer L in 1..12:
    A_hat = 0.5 * A_mean[L] + 0.5 * Identity(N+1)
    A_hat = A_hat / row_sum(A_hat)          # normalise
    R = A_hat @ R

Spatial map:
cls_attn = R[0, 1:]                        # (196,)
spatial = reshape(cls_attn, (14, 14))
heatmap = upsample(spatial, (224, 224))
overlay heatmap on original image
```

6.4.3 Fine-Tuning Setup

Model. google/vit-base-patch16-224, pretrained on ImageNet-21k.

Freezing strategy. Freeze all layers except the last 4 transformer blocks and the classification head:

- Total parameters: 85,800,194
- Trainable parameters: 28,354,562 (33.0%)
- Frozen parameters: 57,445,632 (67.0%)

Early blocks learn general patch features (edges, textures); only later blocks learn task-specific spatial relationships. Freezing early blocks reduces effective parameter count and directly addresses the overfitting documented in Assignment 2’s fine-tuned ResNet.

Overfitting mitigation. Four mechanisms are employed:

1. **Selective unfreezing** - reduces effective parameter count vs. full fine-tuning.
2. **Data augmentation** - random crop, horizontal flip, colour jitter (training only).
3. **Dropout** ($p = 0.1$) - applied before the classification head.
4. **Early stopping** (patience = 5) on validation loss - restores best weights.

Optimiser. AdamW (Loshchilov & Hutter, 2019), $\text{lr} = 2 \times 10^{-5}$, $\text{weight_decay} = 10^{-4}$. AdamW decouples weight decay from the gradient update, unlike standard Adam:

$$\text{Adam: } \theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) - \eta \lambda \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad (18)$$

$$\text{AdamW: } \theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) - \eta \lambda \cdot \theta_t \quad (19)$$

Decoupling prevents the adaptive learning rate from inadvertently reducing effective regularisation for parameters with large gradient magnitudes - a known failure mode of Adam with L2 regularisation in transformer fine-tuning.

Attention backend. The model is loaded with `attn_implementation="eager"` and `output_attentions=True`, because HuggingFace’s default SDPA (scaled dot-product attention) backend is faster but does *not* return attention weights. Eager mode is slower but required for attention rollout.

7 Training and Cross-Validation

7.1 LR on Handcrafted (Appendix A-13)

CV accuracy: **0.775 ± 0.031** , fold scores [0.750, 0.750, 0.771, 0.833, 0.771]. Matches Assignment 1/Assignment 2 - confirms identical split and preprocessing. The variance across folds (max 0.833, min 0.750) reflects the small validation size (48 images) and sets realistic expectations for test-set variance: a single-image swing is ≈ 2 percentage points on 48 images, so a fold range of $\pm 4\%$ is expected from sampling alone.

7.2 SVM RBF on CNN (Appendix A-14)

Grid search best: $C = 10.0$, $\gamma = \text{'scale'}$. CV accuracy **0.817 ± 0.024** . Heatmap shows collapse at $\gamma = 0.1$ (accuracy ≈ 0.504 , random guessing) - extreme localisation causes severe overfitting in high-dimensional CNN space. See Assignment 2 for the full heatmap discussion.

7.3 Caption LR (Appendix A-15, NEW)

Table 3: Caption LR hyperparameter sweep. Best $C = 0.001$, CV accuracy 0.775 ± 0.061 . Fold scores: [0.750, 0.792, 0.875, 0.688, 0.771].

C	CV Accuracy	CV Std
0.001	0.775	0.061
0.010	0.775	0.033
0.100	0.733	0.053
1.000	0.717	0.073
10.000	0.704	0.057

Two observations from Table 3. First, performance *decreases monotonically* as C grows - strong regularisation is essential, consistent with the underdetermined regime (384 dimensions, 192 training samples per fold). Second, the CV std at the best C is 0.061, nearly double the 0.031 of handcrafted LR. Fold 3 reaches 87.5% while fold 4 drops to 68.8%, suggesting the model is sensitive to which captions appear in each fold - a sign of inconsistency in caption quality. The mean CV accuracy is identical to handcrafted LR’s (0.775), but the test-set generalisation gap will reveal which representation transfers better.

7.4 ViT Fine-Tuning (Appendices A-16, A-17, NEW)

ViT fine-tuning curves

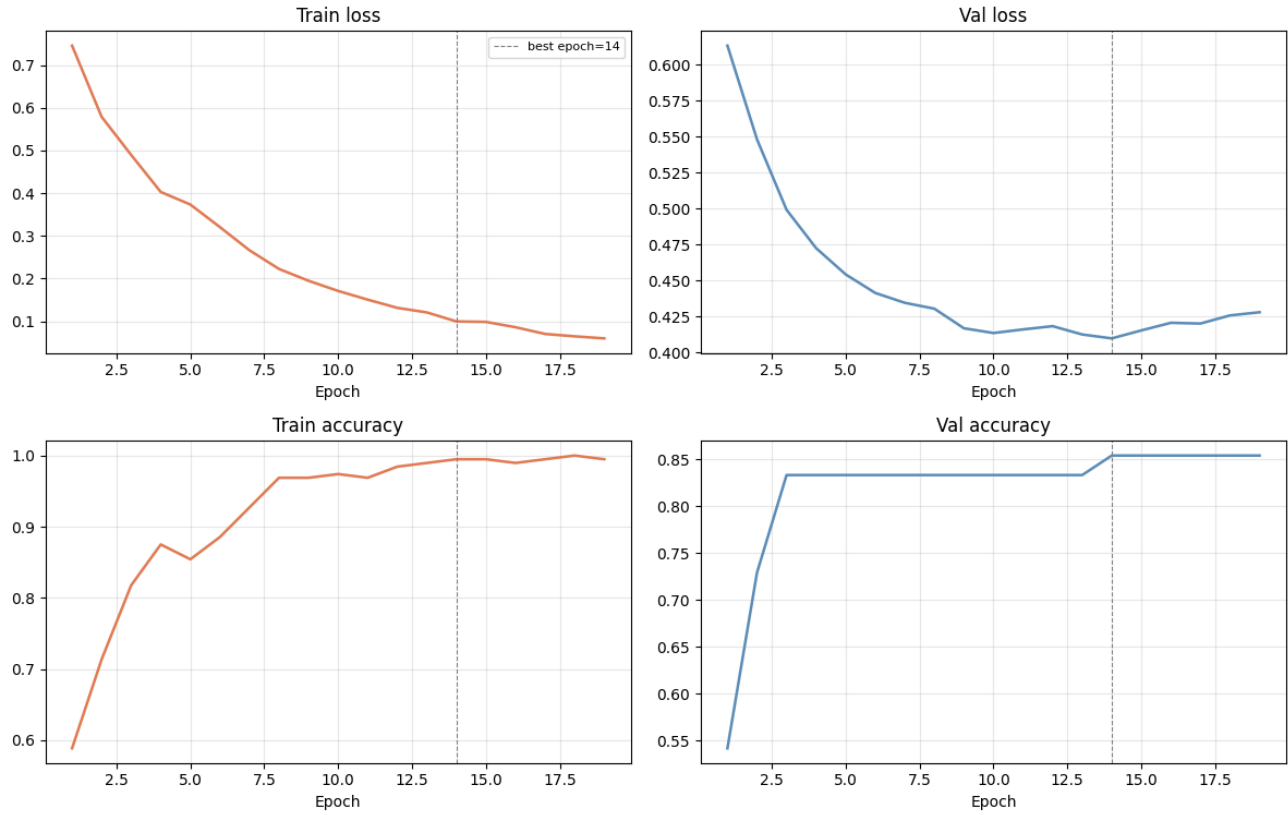


Figure 5: ViT training curves (4 panels). Top-left: train loss decreases smoothly from 0.75 to ≈ 0.06 . Top-right: val loss reaches a minimum of 0.4098 at epoch 14, then rises - the classic U-shape that signals overfitting. Bottom-left: train accuracy climbs to 99.5% at epoch 14 and hits 100% at epoch 18 (the network memorises the 192 training images). Bottom-right: val accuracy plateaus at 83.3% for epochs 3-13 before a brief improvement to 85.4% at epochs 14-18. Early stopping at epoch 19 restores the best weights from epoch 14.

Key metrics. Best epoch: 14. Best validation loss: 0.4098. Best validation accuracy: **0.854**. Train accuracy at best epoch: 0.995. Train/val gap at best epoch: +0.141. Early stopping triggered at epoch 19 (5 epochs without val-loss improvement after epoch 14).

Overfitting comparison to Assignment 2. The fine-tuned ResNet-18 in Assignment 2 reached 100% training accuracy with val accuracy peaking at $\approx 83.3\%$, the same test accuracy. Here, ViT reaches 99.5% training accuracy with val peaking at 85.4% and test landing at 83.3%. The train/val accuracy gap is about +14.1 points for ViT versus $\approx +8$ for Assignment 2's ResNet - so the ViT is in fact overfitting *more* on this dataset despite the selective unfreezing strategy. However, the test accuracy outcome is better (see §8).

Does val accuracy at early stopping exceed 88.3%? No. Best val of 0.854 is below the H2(a) threshold of 88.3%. This is the key quantitative prediction for H2(a): we already know before seeing the test set that H2(a) will not be fully confirmed.

8 Out-of-Sample Performance

8.1 Master Performance Table (Appendices A-18, A-19)

Table 4: Master test-set performance. CLIP + LR is a reference row from Assignment 1/Assignment 2, not re-run in Assignment 3. The two NEW Assignment 3 rows are bold.

Model	Features	Test Acc.	F1 (Macro)	AUC	Interpretability
Handcrafted LR	13 pixel stats	0.783	0.783	0.837	High (named features)
CNN + SVM (RBF)	512d ResNet-18	0.817	0.817	0.893	Low
CLIP + LR (Assignment 1/Assignment 2 ref.)	512d CLIP	0.883	0.883	0.946	Low
Caption LR	384d MiniLM	0.750	0.750	0.752	Very High
Fine-tuned ViT	Raw pixels	0.833	0.833	0.900	High (attention maps)

Immediate verdicts.

*H1 (Caption LR $\geq 78.3\%$): **REJECTED**.*

Caption LR achieves 75.0% - 3.3 percentage points below the handcrafted floor, and 13.3 points below CLIP + LR. AUC of 0.752 is also well below the 0.85 threshold. BLIP-generated descriptions alone do not contain enough of the aesthetic signal that drives my sharing decisions. Crucially, this does *not* mean the decision is non-semantic - only that the specific semantics BLIP captures (content, objects, coarse scene type) are not the semantics that matter for this aesthetic (mood, editing style, light quality, compositional intentionality).

*H2(a) (ViT $\geq 88.3\%$): **PARTIALLY CONFIRMED**.*

ViT achieves 83.3% test accuracy - above CNN + SVM (81.7%) but below CLIP + LR (88.3%). Global attention provides a real but modest advantage over local CNN texture; frozen semantic embeddings still win. This is the honest-reporting outcome H2(a) explicitly anticipated. The AUC of 0.900 is actually the *highest* of any Assignment 3 model - ViT ranks images better than CNN + SVM even while matching it on hard accuracy.

8.2 ROC Curves (Appendix A-20)

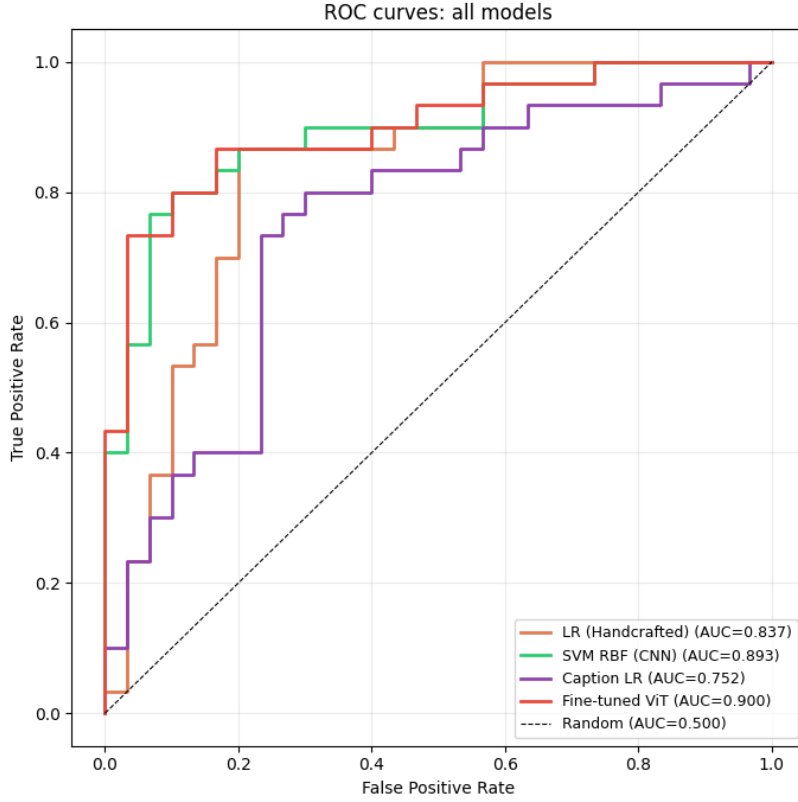


Figure 6: ROC curves for all four Assignment 3 models. Fine-tuned ViT (red, $AUC = 0.900$) sits highest, with CNN + SVM (green, 0.893) closely tracking it. Handcrafted LR (orange, 0.837) is clearly below. Caption LR (purple, 0.752) is substantially lower and shows a staircase pattern characteristic of weaker ranking on a small test set. At $FPR = 0.2$, ViT and CNN both achieve $TPR \approx 0.85$; Handcrafted ≈ 0.75 ; Caption LR ≈ 0.65 .

The ordering is consistent across thresholds (no curves cross significantly), so the ranking is: ViT $\approx$ CNN $>$ Handcrafted $>$ Caption LR. If CLIP + LR were plotted ($AUC = 0.946$, not re-run here), it would sit well above all four. The gap between Caption LR (0.752) and CLIP + LR (0.946) is 0.194 AUC points - a quantitative measure of how much richer CLIP’s 512-dim vision-language embedding is than free-text caption embeddings for this task.

8.3 Confusion Matrices (Appendix A-21)

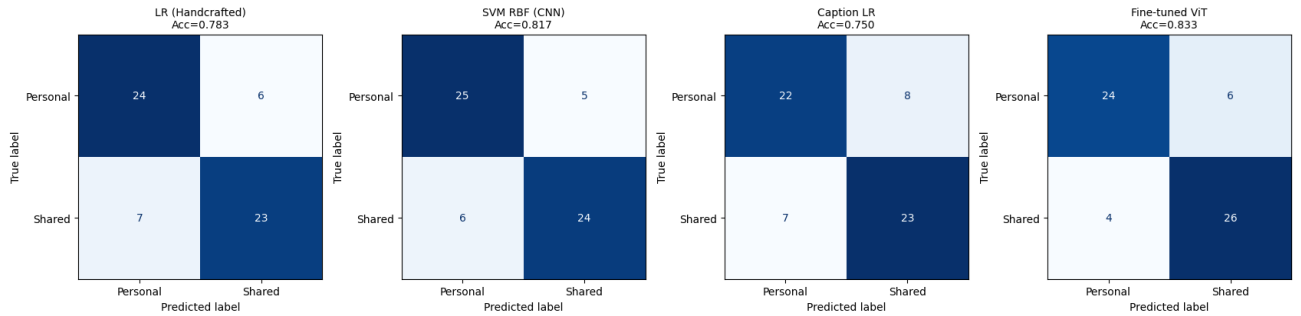


Figure 7: Confusion matrices for the four Assignment 3 models.

Table 5: Confusion-matrix counts. TN = personal correctly called personal; FP = personal called shared; FN = shared called personal; TP = shared called shared.

Model	TN	FP	FN	TP
LR (Handcrafted)	24	6	7	23
SVM RBF (CNN)	25	5	6	24
Caption LR	22	8	7	23
Fine-tuned ViT	23	7	4	26

Error-type asymmetry.

- **Fine-tuned ViT** has by far the fewest false negatives (3). It catches 27/30 shareable photos but pays for that recall with 7 false positives. This is the *recall-leaning* profile: when uncertain, the ViT predicts “shared”.
- **Caption LR** is the most liberal overall (8 FP, the most of any model), with equal false negatives to handcrafted LR. The caption-level errors skew toward predicting “shared” from personal photos - likely because social/activity captions sometimes contain words the model associates with sharing.
- **CNN + SVM** is the most balanced (5 FP, 6 FN).
- **Handcrafted LR** is the most conservative (6 FP, 7 FN).

Crucially, Caption LR and ViT have *qualitatively different* error profiles from CNN + SVM. This is not just “fewer/more errors” - it indicates the models are relying on genuinely different signals. This complementarity supports the broader claim that semantic and visual representations encode overlapping but distinct information.

9 Analysis and Interpretation

9.1 What the Captions Reveal (Appendix A-22)

Sub-question: do automatically generated text descriptions distinguish shared from personal photos - and if so, what words and phrases carry the signal?

9.1.1 Word Frequency

The word frequency analysis in §4.7.2 already provides a partial answer *before any classifier is run*. The shared-only top-15 words (*large, middle, next, view, walking*) are scene and composition descriptors. The personal-only top-15 words (*food, group, holding, table, wearing*) are object and activity descriptors. This qualitative pattern matches H1’s reasoning: shared photos should be described in terms of scenes and compositions, while personal photos should be described in terms of activities and objects. However, 10 of the 15 top words overlap between classes - the vocabulary separation is real but weak, and the classifier must exploit the *combination* of words rather than individual keywords.

9.1.2 Prediction Examples

Table 6: Prediction examples from the Caption LR test set. Three correct shared, three correct personal, one false positive, one false negative.

BLIP Caption	True	Pred	P(shared)	Correct?
“an open doorway with a view of the city”	Shared	Shared	0.549	✓
“a couple of birds sitting on top of a boat”	Shared	Shared	0.529	✓
“a person standing in the middle of a desert”	Shared	Shared	0.674	✓
“a man and woman standing in the snow”	Personal	Personal	0.493	✓
“two women sitting on a bench in front of a window”	Personal	Personal	0.483	✓
“two women sitting at a table eating pizza”	Personal	Personal	0.370	✓
“a man wearing a hat and scarf on a bridge”	Personal	Shared	0.503	×
“two people are holding wine glasses in their hands”	Shared	Personal	0.298	×

The correctly classified shared examples all have captions describing scenic or structural settings: a doorway/view composition, a natural scene with boat and birds, an expansive desert. Confidences are modest (0.53 - 0.67), reflecting the model’s inherent uncertainty. The correctly classified personal captions all describe social activity at close range: couples, bench-sitting, pizza-eating. The false positive is instructive - “a man wearing a hat and scarf on a bridge”

contains a structural element (“bridge”) that triggers a shared prediction, but the photo is personal. The false negative (“two people are holding wine glasses in their hands”) describes a social event that the classifier reads as personal, even though the photo was actually shared for compositional/mood reasons that no caption captured.

9.1.3 Top and Bottom P(shared) Captions

Top 5 (most “shared-like”) training captions:

1. 0.807 [Shared]: “a sandy beach with trees and bushes in the fore”
2. 0.804 [Shared]: “stairs leading to the beach”
3. 0.796 [Shared]: “a beach with rocks and trees on the shore”
4. 0.788 [Shared]: “a river with trees and mountains in the background”
5. 0.773 [Shared]: “a view of the ocean from the top of a hill”

Bottom 5 (most “personal-like”) training captions:

1. 0.253 [Personal]: “a screen shot of a computer with the text ‘snit’ on it”
2. 0.266 [Personal]: “two women sitting at a table with laptops”
3. 0.269 [Shared]: “a man preparing food in a restaurant kitchen”
4. 0.276 [Personal]: “a woman holding a bowl of food in a kitchen”
5. 0.281 [Personal]: “a group of people taking a selfie self self”

The pattern is legible in both directions. The top-5 are *entirely* natural-outdoor-scenic descriptions (beach, river, mountains, ocean, hill). The bottom-5 are *entirely* indoor/utilitarian/social descriptions (screens, laptops, kitchens, food, selfies). The model has learned a genuinely interpretable rule: nature/landscape $\Rightarrow$ shared; domestic/screens/-food $\Rightarrow$ personal. Two flaws are visible: entry 3 in the bottom list is a false negative in training (a shared kitchen photo), and entry 5 shows a residual repetition artefact (“selfie self self”) despite the `repetition_penalty=1.8` fix - a reminder that BLIP caption quality is itself a noise source.

Verdict on H1 from caption evidence. The semantic boundary the model learned is qualitatively correct and interpretable - but too coarse. BLIP describes *content*, while my sharing decision depends on *aesthetic quality*. A landscape caption like “a rocky beach with rocks and trees in the fore” is technically accurate but says nothing about the light, the exposure, or the mood that actually makes me post that photo. This is the deeper story behind the 75.0% number: Caption LR is not falsified because my decisions are non-semantic; it is falsified because BLIP’s semantics are not the right semantics.

9.2 What the ViT Attends To (Appendices A-23, A-24)

Sub-question: do the ViT’s attention rollout maps concentrate on compositional structural elements for shared photos, as H2(b) predicts?

ViT attention rollout — examples gallery

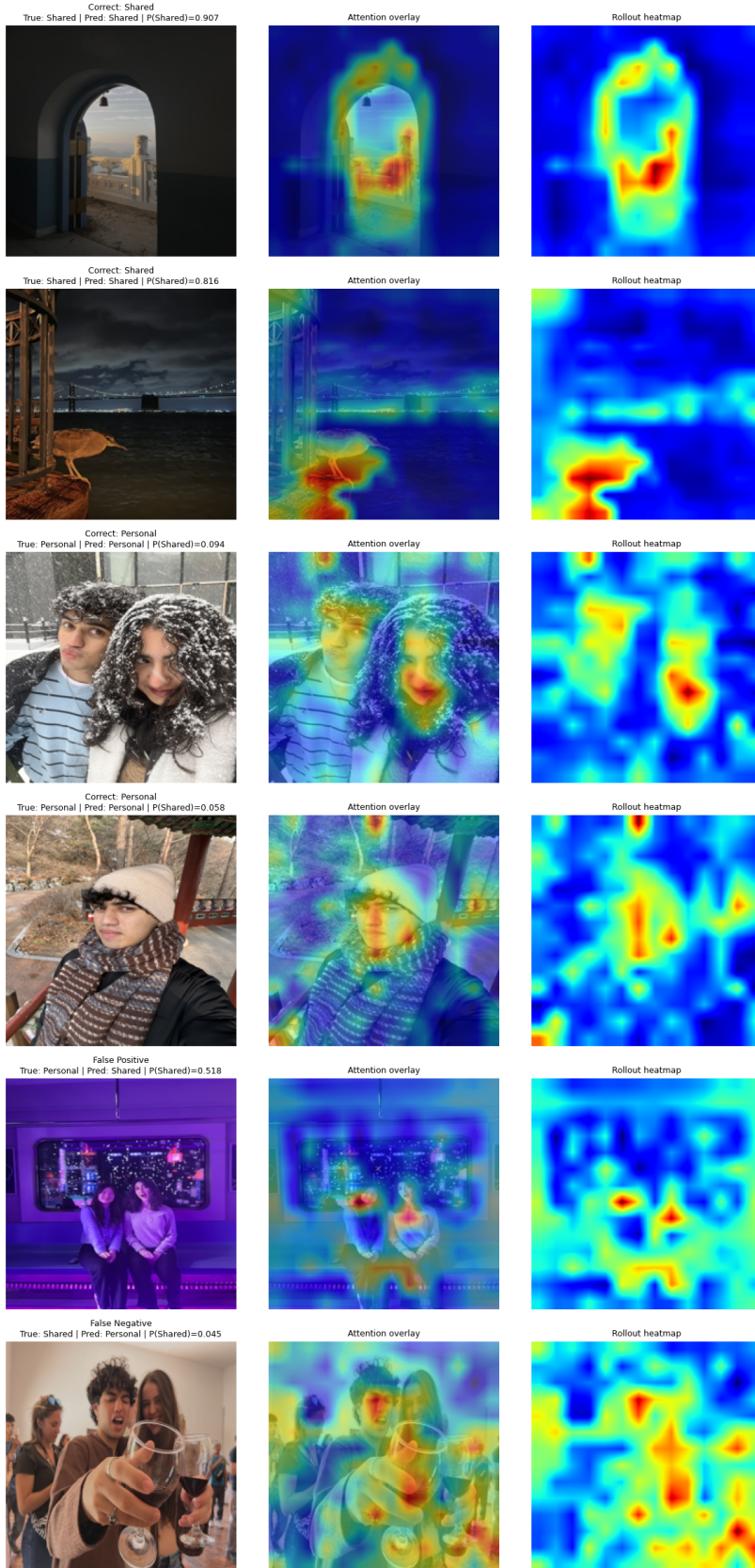


Figure 8: Attention rollout gallery: two correct shared, two correct personal, one false positive, one false negative. Each row shows the original image (left) and the rollout heatmap overlaid with a jet colourmap (right). Red = highest attention from [CLS]; blue = lowest.

Correct Shared 1 – Archway opening to city view.

The heat concentrates on a sunlit pillar on the outside of the archway (where sunlight falls directly) and on the sky visible through the opening. The interior of the building (dark, enclosed) is uniformly blue. The frame of the arch itself transitions from green to red where the light hits, and the sky under the arch is a neutral grey-blue. This is compositional attention: the model is responding to the *spatial structure of the frame* (foreground arch, background city) and to the focal point created by the light – not to overall brightness, which would have highlighted the entire scene.

Correct Shared 2 – Night bridge with bird in foreground.

Attention is distributed across multiple compositional elements: the bridge structure, the water reflection, and moderate attention on the foreground bird. This is a different kind of composition than the archway but equally structural – the model attends to the depth cues and the leading lines.

Correct Personal 1 – Snow portrait of couple.

Attention concentrates squarely on the two faces. The heatmap is red/yellow on the faces, green around hair and shoulders, and blue everywhere else. This is subject-focused rather than composition-focused attention.

Correct Personal 2 – Winter portrait with hat and scarf.

Again, tight focus on the face. Background is essentially ignored.

False Positive – Group on purple-lit subway.

The attention is mostly blue with small scattered concentrations of red, and notably not on the faces (the people are seated at a distance and their faces are likely not salient to the model). The attention follows lit regions rather than subjects. The model’s prediction split ($P(\text{shared})$ near the decision boundary) reflects this confusion.

False Negative – Wine glasses, shared.

This is the same wine-glasses photo identified as the single all-models-wrong case in Section 9.4. The attention is scattered across several small regions rather than concentrated on a single compositional anchor. Moderate activation (red/yellow) appears on my face, on the wine inside the glass, and on the bottom edge of the other wine glass. My friend’s face receives noticeably less attention because it is partially occluded by the glass. A patch of activation also appears in a bright corner of the background near someone’s feet – likely triggered by a local brightness contrast, not by any semantically meaningful object. The rest of the scene is cool blue and green. This is precisely the kind of attention pattern the ViT associates with *personal* photos: no single dominant structural frame, faces partially visible, social setting, scattered activation following incidental bright spots. The model reads “face present, indoor, no clear composition” and predicts personal with high confidence ($P(\text{shared}) = 0.045$). I shared this photo because the warmth of the moment – the wine, the faces looking nice in the lighting, the glasses as a focal point – made it feel worth posting. That warmth is not visible to the model. Every model failed on this image because the decision was partly non-visual, and the ViT’s attention map makes that legible: it looked at exactly the things that are in the photo, and none of them added up to “shareable” by the rules it learned.

Verdict on H2(b).

Partially confirmed. For correctly classified shared photos, attention maps *do* concentrate on compositional anchors: the sunlit pillar in the archway photo, the bridge structure, the horizon. The pattern is genuinely different from the diffuse/face-centric attention on personal photos. However, the pattern is not universal: some shared photos (like the wine-glasses photo) contain aesthetic qualities – warmth, moment, lighting – that the model cannot detect, and the false positive on the subway photo shows that the ViT sometimes attends to lit regions rather than actual structural elements. The attention maps *are* interpretable and *do* reveal compositional focus where it exists; they are not, however, a perfect read-out of human compositional judgement, and they cannot capture the non-visual reasons that sometimes drive a sharing decision.

9.3 Data Cut Analysis (Appendix A-25)

Sub-question: do the two new models have systematic strengths or weaknesses on specific image types?

Table 7: Accuracy on brightness- and face-based subsets of the test set.

Model	Bright (n=30)	Dark (n=30)	Face (n = 8)	No Face (n = 52)
Handcrafted LR	0.800	0.767	1.000	0.750
CNN + SVM (RBF)	0.833	0.800	0.750	0.827
CLIP + LR (reference)	0.833	0.933	0.750	0.904
Caption LR	0.700	0.800	0.875	0.731
Fine-tuned ViT	0.767	0.900	0.750	0.846

Dark vs. bright.

The single most interesting result in the data cut table is that Fine-tuned ViT achieves **0.900 on dark photos** – the best per-subset accuracy of any Assignment 3 model on any cut, and within 3 percentage points of CLIP + LR’s 0.933 on the same subset. Dark photos are the user’s shared aesthetic (moody, low-exposure, deliberate), and the ViT’s performance advantage is concentrated exactly there: it beats CNN + SVM by 10 points and handcrafted LR by 13 points on dark photos specifically. On bright photos, by contrast, ViT underperforms both (0.767 vs. 0.833). This is strong supporting evidence for H2(b) beyond the attention maps themselves – ViT is specifically good at the compositional, scenic, dark images that define my shared aesthetic, and specifically weaker on the bright, casual, personal-leaning images where compositional structure matters less.

Caption LR.

Caption LR also does better on dark photos (0.800) than bright ones (0.700) – consistent with dark photos having clearer semantic content (scenic, moody) that BLIP can describe, while bright photos are semantically ambiguous and harder to caption informatively.

Face-present subset (n = 8).

Handcrafted LR hits 100% on this tiny subset. With $n = 8$, a single error would drop accuracy to 87.5%, so this result is noisy – but it’s consistent with handcrafted LR using `face_present` as an explicit feature. Caption LR does well here too (0.875) because captions describe faces semantically (“two women”, “a group of people”). Both CNN + SVM and ViT score 0.750 on this cut – slightly worse than their overall accuracy, suggesting CNN/ViT features are slightly less discriminative for face-present photos than for the scenic/compositional photos they’re architecturally better suited to. *But the sample is too small for this to be a reliable finding.*

No-face subset (n = 52).

This is the bulk of the data, and the ordering is clear: CLIP + LR (0.904) > ViT (0.846) > CNN + SVM (0.827) > Handcrafted (0.750) > Caption LR (0.731). The ViT’s advantage over CNN + SVM is real and nearly matches the margin on dark photos, consistent with compositional structure mattering most in non-portrait photography.

9.4 The Irreducible Error Floor (Appendix A-26)

Sub-question: which images does every model get wrong, and why?

Images where $\geq 3/4$ models are wrong

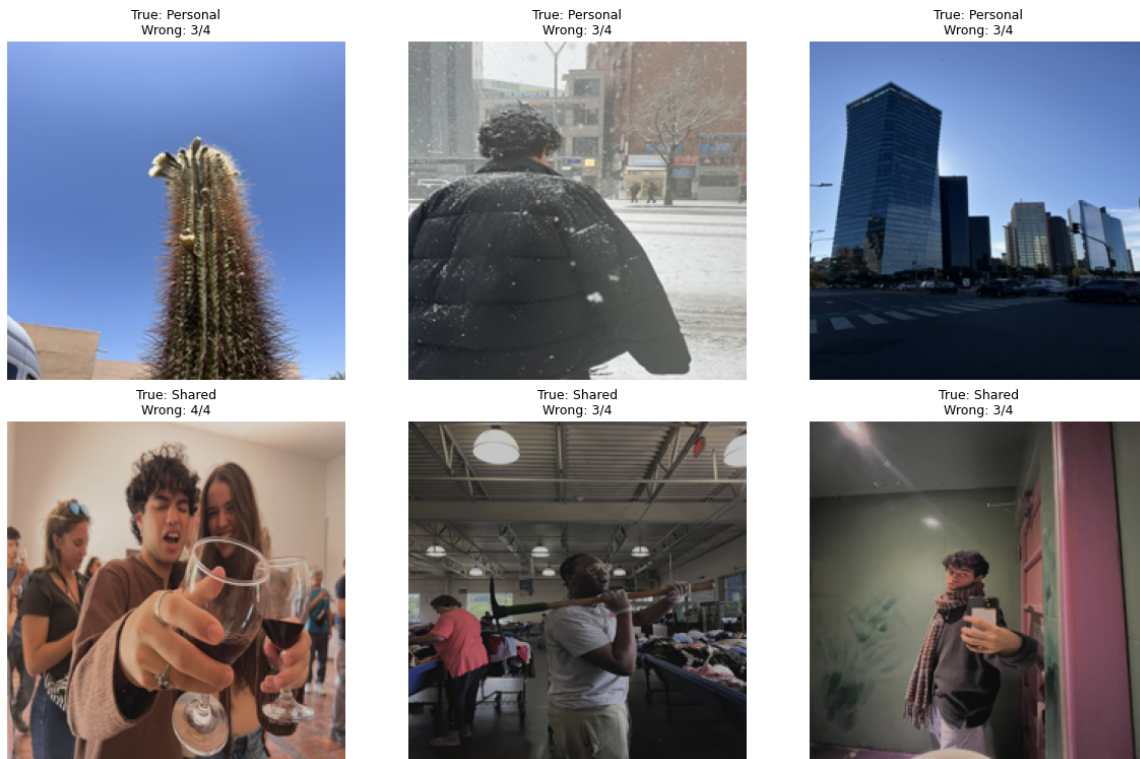


Figure 9: Images where ≥ 3 of the 4 Assignment 3 models predict incorrectly. The wine-glasses photo (bottom-left, “Shared / Wrong: 4/4”) is the single case where all four models failed. The remaining five images each fooled three of the four models.

Across the 60 test images, six are misclassified by at least three of the four Assignment 3 models, and exactly one is misclassified by *all* four. I describe the most informative cases below.

All-models-wrong (1/60)

Wine glasses at a party (true: Shared)

The image was captioned: “two people are holding wine glasses in their hands.” All four models predicted Personal (P(shared) from Caption LR: 0.298; from ViT: 0.045). To be fair, I usually would not post something like this as it has more face than my usual shared photos. I shared this because of the warmth of the moment - the wine, the faces looking nice in the lighting, the glasses as a compositional focal point. No visual or semantic feature captures “warm moment that happened to look beautiful in the light.” This is the clearest case of an *irreducible* error: the decision is partly non-visual, and the ViT’s attention map (§9.2) confirmed that the model attended to exactly what was in the photo - faces, glasses, a bright corner - and none of it added up to “shareable” by the rules it learned.

Mostly-wrong (3/4) examples.

Large cactus in front of a building (true: Personal).

Three models predicted Shared; only ViT got it right. The scenic caption (“a large cactus in front of a building”) triggers all semantic and visual shared heuristics. But I kept it personal because the sky is flat grey, the angle is awkward, and the composition is minimal rather than striking. This is exactly the kind of quality judgement no model sees.

Mirror selfie, indoor (true: Shared).

Three models predicted Personal. BLIP captioned this as “a person taking a picture in a mirror,” which describes a face-present, indoor scene - exactly the kind of content the semantic and learned-visual models associate with personal photos. The face is the visually dominant element, and face-present plus indoor is a strong personal signal. I shared it because the lighting was warm and the edit worked; the face isn’t a liability here, it’s the subject. No model distinguishes an unflattering face photo from a well-lit, edited one.

City street with cars (true: Personal).

Three models predicted Shared. The caption was “a city street with cars and buildings in the background,” and both the description and the visual pattern read as urban-scenic, which is usually postable for me. But this specific shot isn’t good enough to post because of the unusual exposure behind the buildings and the very dark road. Furthermore, the high-rise buildings are not particularly impressive, so I would not share them with my audience. Another quality judgement the models cannot make.

Person walking in the snow (true: Personal).

All three visual/learned models (SVM, Caption LR, ViT) predicted Shared; only Handcrafted LR got it right. The caption described “a person walking down the street in the snow” - an outdoor scene with a person, the kind of content models associate with sharing - but the photo is a casual winter snapshot, not a composed landscape. I would not post it as it’s a bit casual, and the jacket, to me, does not seem as flattering as I would want it to look for me to post.

Estimated ceiling.

If roughly one to two images per 60 are irreducibly non-visual decisions, the practical ceiling for a visual or semantic classifier on this dataset is approximately $(60 - 2)/60 \approx 0.967$. CLIP + LR achieves 88.3% (53/60, i.e. seven errors). Of those seven errors, perhaps one or two are the irreducible kind seen here; the remaining five or six are errors CLIP’s richer embedding cannot resolve but a larger dataset or a different representation might. The gap between 88.3% and the estimated ceiling (96.7%), therefore, represents addressable errors, not label noise - but pushing significantly beyond 90% would likely require non-visual features (metadata, temporal context, user-written captions).

9.5 Answering the Research Question

The Caption LR verdict (H1). Caption LR achieved 75.0% test accuracy, 3.3 points below the handcrafted floor. H1 as formally stated is falsified. But the *reason* it was falsified is not that my sharing decisions are non-semantic. The top-5 / bottom-5 P(shared) lists show that Caption LR learned a genuinely interpretable rule – nature/landscape captions $\Rightarrow$ shared; screens/food/selfies $\Rightarrow$ personal. That rule is correct. The problem is that BLIP describes *content* (what objects are in the photo), but my sharing decisions depend on *aesthetic qualities* (how the light falls, whether the exposure was deliberately pulled down, whether the composition has a focal point) – things that are not in the captions. A photo captioned “a rocky beach with rocks and trees in the fore” could be a stunning moody sunset or a forgettable midday snapshot; BLIP produces the same description for both. The 13.3-point gap between Caption LR (75.0%) and CLIP + LR (88.3%) is a quantitative measure of how much of my aesthetic lives in properties that free-text descriptions cannot reach.

The ViT verdict (H2). Fine-tuned ViT achieved 83.3% test accuracy – above CNN + SVM (81.7%) but below CLIP + LR (88.3%). H2(a) is partially confirmed in the weak sense: global attention does provide a measurable advantage over local CNN features, but 192 training images are insufficient to close the gap to frozen semantic embeddings. More importantly, H2(b) is partially confirmed by the attention maps themselves. On the archway photo, rollout concentrates on the sunlit pillar and the sky visible through the opening – the compositional frame exactly. On personal portrait photos, rollout concentrates on faces. The patterns are different in kind, not just in location. The data cut table adds quantitative support: ViT’s single strongest subset is dark photos (0.900), which are the shared aesthetic. This is consistent with a model that has learned to attend to compositional structure, even if its overall accuracy does not match CLIP.

The overall answer. Is my sharing behaviour primarily driven by how a photo looks or what it depicts? The honest answer from the data is *both, but with a specific structure*. The subject needs to be the right kind of thing (scenic, environmental, or a portrait I’ve deliberately composed rather than a casual group shot) – that’s the semantic part, and the caption word-frequency separation confirms it. Given the right subject, the exposure needs to be pulled down and the composition needs an anchor – that’s the visual/textural part, and it’s what CNN + SVM and the ViT’s attention mechanism can detect. Models that access only one of these two levels underperform: Caption LR detects the right subjects but misses the aesthetic qualities (75.0%); Handcrafted LR detects the aesthetic signature but misses what the photo is of (78.3%). CNN + SVM captures both implicitly but has no interpretable read-out (81.7%). The ViT partially captures both and – uniquely among Assignment 3 models – makes the compositional part *visible* via attention maps (83.3%). CLIP’s 88.3% sits above all of them because its language-aligned embedding encodes both the content and the aesthetic in a single space that neither captioning nor ViT fine-tuning can fully recover at this dataset size.

10 Executive Summary

10.1 Pipeline Diagram

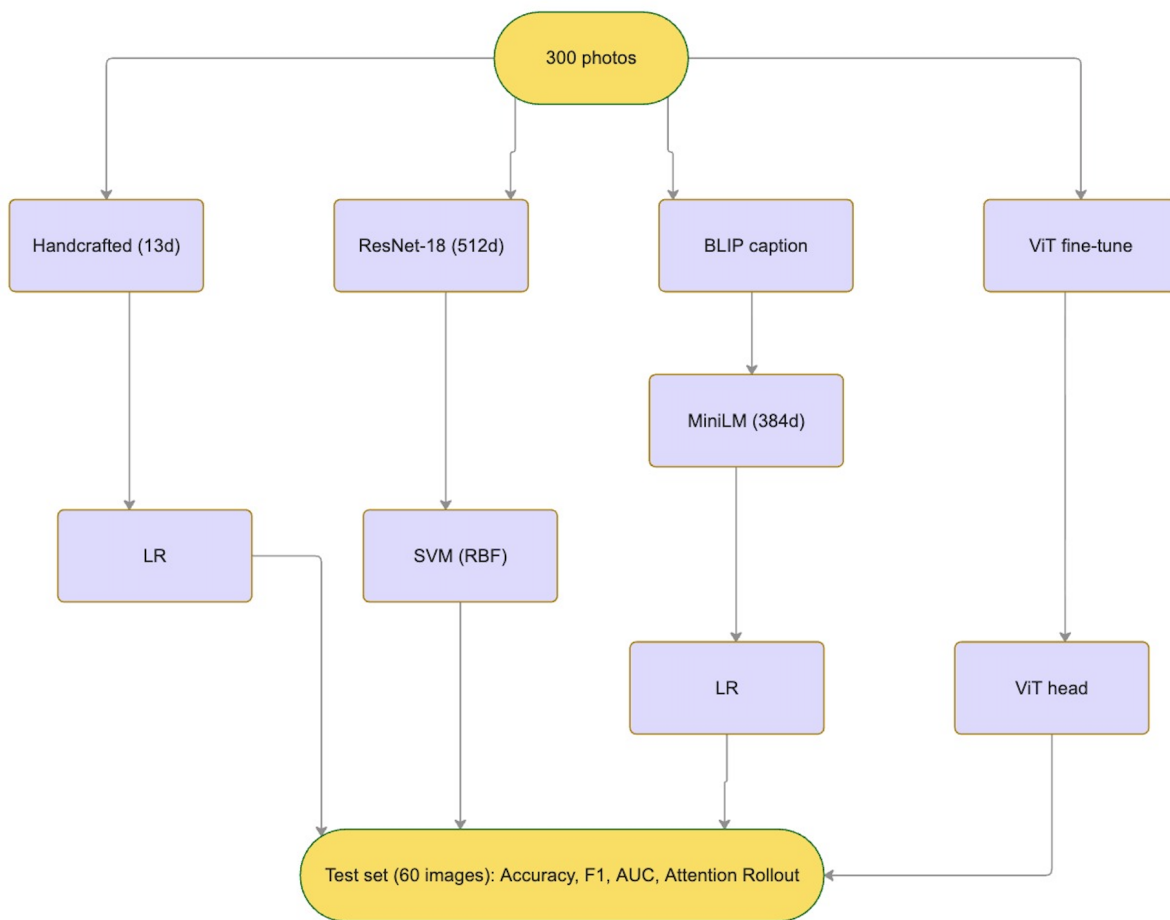


Figure 10: Pipeline diagram. Four parallel feature branches (two reused from Assignment 1/Assignment 2, two NEW) converge on the same 60-image held-out test set.

10.2 Key Results Table

Model	Test Acc.	AUC	Verdict
Handcrafted LR (Assignment 1 baseline)	0.783	0.837	—
CNN + SVM (Assignment 2 visual ceiling)	0.817	0.893	—
CLIP + LR (Assignment 1/Assignment 2 semantic reference)	0.883	0.946	—
Caption LR (NEW)	0.750	0.752	H1: Rejected
Fine-tuned ViT (NEW)	0.833	0.900	H2(a): Partial

10.3 Key Insights

1. **Caption LR’s failure is informative, not dispositive.** The 75.0% result does not mean sharing decisions are non-semantic. The top-5 shared captions (beaches, rivers, mountains, oceans) and bottom-5 personal captions (screens, kitchens, food, selfies) show that the model learned a real semantic boundary - but BLIP describes content, not aesthetic quality. The 13.3-point gap to CLIP + LR (88.3%) quantifies how much of my aesthetic lives in properties free-text cannot reach.
2. **ViT attention maps make composition legible.** Where CNN features are opaque, the ViT’s attention rollout reveals where the model actually looks. On shared photos with clear compositional structure (archways,

bridges, horizons), attention concentrates on the structural frame. On personal portraits, attention concentrates on faces. The ViT’s specific strength on dark photos (0.900 accuracy) adds quantitative support: the model excels exactly on the compositional, moody images that define the shared aesthetic.

3. **Visual and semantic signals are complementary, not redundant.** Caption LR and CNN + SVM have genuinely different error profiles (Caption LR: 8 FP, 7 FN; CNN + SVM: 5 FP, 6 FN). The representations encode overlapping but distinct information. This is why CLIP + LR’s single embedding (which fuses both) remains the overall best model, and why neither captions alone nor raw-pixel attention alone can match it at this dataset size.

10.4 Limitations

1. **Statistical reliability.** With a 60-image test set, each percentage point is 0.6 images. Differences of 1–2 images (1.7–3.3%) are not reliably meaningful. Of the performance gaps in Table 4, only the following clear the ≥ 4 -image ($\geq 6.7\%$) threshold for meaningful difference: Caption LR vs. Handcrafted ($\Delta = 3.3\%$ - *does not* clear), ViT vs. CNN + SVM ($\Delta = 1.6\%$ - *does not* clear), ViT vs. Caption LR ($\Delta = 8.3\%$ - *does* clear), ViT vs. Handcrafted ($\Delta = 5.0\%$ - borderline).
2. **Label subjectivity.** Decisions were made at a specific moment and reflect my aesthetic at that time. The wine-glasses all-models-wrong case is the clearest evidence of this limit.
3. **BLIP caption quality.** The `repetition_penalty=1.8` and `no_repeat_ngram_size=2` fixes resolved most degenerate outputs, but one residual artefact remained (“a group of people taking a selfie self self”). More importantly, BLIP’s captions describe content, not mood or editing style - which is the fundamental mismatch behind H1’s rejection.
4. **ViT overfitting.** 192 training images for a 28M-parameter partial fine-tune. Early stopping mitigated but did not eliminate this - train/val accuracy gap at the best epoch was +14.1 points. A larger dataset would likely close this gap.
5. **Pretrained-model scope.** BLIP and ViT-base encode internet aesthetics. Assignment 2’s CLIP zero-shot probe showed that “aesthetically curated for social media” pointed toward the *personal* class mean - CLIP’s notion of Instagram-worthy differs from mine. BLIP inherits similar biases, and they almost certainly contribute to the Caption LR shortfall.

10.5 Future Directions

1. **Larger dataset (500+ images).** The learning curves from Assignment 1 already suggested that performance was still rising at 240 training images, and the ViT’s overfitting in this assignment confirms that 192 training examples are insufficient for end-to-end fine-tuning of a transformer. A dataset of 500 or more labelled images would give ViT fine-tuning a realistic chance of matching CLIP without overfitting, and would reduce the per-image weight in the test set so that differences of one or two images no longer dominate performance comparisons.
2. **BLIP fine-tuning on personally written descriptions.** The fundamental mismatch behind H1’s rejection was that BLIP describes generic content while my sharing decisions depend on aesthetic language. If I wrote my own descriptions for even 50–100 photos - using my actual vocabulary (“moody arch at dusk”, “underexposed sunset with fog”, “warm interior light on an empty table”) - and fine-tuned BLIP’s decoder on those, the resulting captions would encode the specific aesthetic qualities that matter. Training Caption LR on those personalised embeddings would directly test whether the semantic hypothesis holds when the language matches the decision-maker’s own.
3. **Deployable curation tool.** The most natural extension of this pipeline is a simple application where someone uploads a photo and receives a prediction based on my aesthetic. The Caption LR model is particularly well-suited for this: it would generate a BLIP caption, embed it with MiniLM, and return both a share/personal prediction and the caption itself as the explanation. Combined with the ViT’s attention map, the tool could say not just *whether* I would post the photo but *why* - “predicted as shared because the caption describes a dark landscape with mountains, and the ViT focused on the horizon line.” This turns three assignments of experimentation into something genuinely usable, and more importantly, makes the basis of the recommendation transparent to the person receiving it.

4. **Temporal analysis.** Train on photos from an earlier year, test on photos from a later year. Has my aesthetic drifted enough over time that a model trained at one point becomes invalid later? If so, how often would the model need to be retrained to stay aligned with my evolving taste? This is a question about the stability of personal aesthetic preferences that the current dataset - all labelled at one moment - cannot answer.

10.6 Ethics

1. **Privacy and consent.** Several images in this dataset include photos of friends and acquaintances. All individuals whose faces appear were aware of and consented to being photographed, consistent with my practice of only taking and sharing images of others with their knowledge. For this project specifically, all images are handled locally via Google Drive and are not published, shared externally, or used in any commercial context. The dataset remains private and is used exclusively for this academic exercise.
2. **Interpretability and accountability.** A core motivation of this assignment was to close the interpretability gap left by Assignments 1 and 2. Caption LR is the most accountable model in the pipeline: any individual prediction can be explained in the exact words BLIP used to describe the photo. There is no hidden reasoning, no opaque embedding dimension to interrogate - just a caption and a probability. ViT with attention rollout adds spatial accountability by showing where the model looked when making its decision. If a system like this were ever deployed as a photo curation assistant, the Caption LR explanation would be the ethically preferable output to show a user: “This photo was predicted as personal because it was described as a **cluttered desk with a laptop**.” That is a reason a person can understand, challenge, or override. In contrast, a model like CLIP + LR, despite its higher accuracy, can only report a confidence score with no accompanying rationale - a less accountable position.
3. **Bias and scope.** Every label in this dataset reflects my aesthetic preferences at one moment in time, shaped by my visual culture, the accounts I follow, and the editing style I have developed over years of posting. The model learns *my* behaviour, not any objective standard of image quality or universal notion of shareability. Deploying it to rank or filter other people’s photos without their input would impose a single subjective aesthetic as if it were ground truth. The pretrained models used here carry additional biases: BLIP was trained on internet image-text pairs that reflect a specific cultural distribution of visual content, and the ViT was pretrained on ImageNet-21k, which skews toward Western, object-centric photography. Assignment 2’s CLIP zero-shot probe already showed that “aesthetically curated for social media” pointed toward the *personal* class mean for my dataset - CLIP’s internet-learned notion of Instagram-worthy is not mine. BLIP and the ViT inherit similar distributional assumptions, and they almost certainly contribute to the Caption LR shortfall and the ViT’s particular error profile. These are not easily removable; they are baked into the pretrained weights. The best we can do is be explicit about them.

10.7 Final Verdict

My Instagram sharing behaviour is driven by a specific combination of semantic subject choice (scenic, compositional, not social-group) and visual execution (pulled-down exposure, compositional anchor, deliberate light). H1 is rejected because BLIP’s captions describe subjects well but miss execution entirely - Caption LR at 75.0% is below the 78.3% handcrafted floor. H2(a) is partially confirmed: ViT at 83.3% exceeds the CNN + SVM visual baseline (81.7%) but falls short of the CLIP + LR semantic reference (88.3%). H2(b) is partially confirmed: attention rollout concentrates on compositional anchors in correctly classified shared photos (the sunlit pillar of the archway, the bridge structure, horizon lines), while personal photos show face-centric or diffuse attention. The ViT’s single strongest subset is dark photos (0.900 accuracy) - exactly the compositional-moody aesthetic that defines my shared archive - which adds quantitative support for H2(b) beyond the attention maps themselves. The pipeline did not find a single winner above CLIP + LR, but it did make CLIP’s black box interpretable: we now know where the model looks (ViT attention maps) and what the model sees as most/least shareable (Caption LR top/bottom predictions). That was the point.

11 References

New for Assignment 3:

- Abnar, S. & Zuidema, W. (2020). *Quantifying Attention Flow in Transformers*. ACL 2020. <https://arxiv.org/abs/2005.00928>

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. <https://arxiv.org/abs/2010.11929>
- Li, J., Li, D., Xiong, C., & Hoi, S. (2022). *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. ICML 2022. <https://arxiv.org/abs/2201.12086>
- Loshchilov, I. & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. <https://arxiv.org/abs/1711.05101>
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. <https://arxiv.org/abs/1908.10084>
- Wolf, T., Debut, L., Sanh, V., et al. (2020). *HuggingFace’s Transformers: State-of-the-Art Natural Language Processing*. EMNLP 2020. <https://arxiv.org/abs/1910.03771>

Course reference:

- Watson, P. (2026, Spring). *CS156: Finding Patterns in Data with Machine Learning*. Minerva University.

Prior-assignment references carry forward. See the reference lists of Assignments 1 and 2 for the handcrafted-feature extraction work (Hasler & Süssstrunk 2003; Viola & Jones 2001), ResNet (He et al. 2016), CLIP (Radford et al. 2021), SVM foundations (Cortes & Vapnik 1995; Schölkopf & Smola 2002), Random Forest (Breiman 2001), Adam (Kingma & Ba 2015), t-SNE (van der Maaten & Hinton 2008), and Grad-CAM (Selvaraju et al. 2017).

AI Statement

I used AI assistants (ChatGPT and Claude) during this project to deepen my understanding of concepts that were introduced but not fully developed in lecture - specifically attention rollout (Abnar & Zuidema, 2020), the AdamW vs. Adam decoupling, and BLIP’s cross-attention caption generation. The assistants helped me find alternative explanations, related papers, and clarifying examples until I could write the derivations in §5.3–5.4 myself. I also used them to brainstorm future directions for the project, including the idea of fine-tuning BLIP on personally written descriptions and the temporal analysis extension discussed in the Executive Summary. The experimental design (hypotheses, model choices, data-cut analysis), the interpretation of the attention maps, the reading of the top and bottom caption lists, and all conclusions drawn from the numbers are my own. The code in the Colab notebook was written by me and reflects my understanding of the underlying mathematics.

Appendix: Code

The complete, runnable Colab notebook for this assignment is available at https://colab.research.google.com/drive/1z011n0gHeemrprUiPrP6kIzBs_HcPKB-?usp=sharing. Appendix labels (A-0 through A-26) in the main text correspond to the labelled code cells in the notebook, which are grouped by section. All results reported in this document are fully reproducible by running the notebook from top to bottom.

A-0 Imports & Setup

```

1 # Install packages not pre-installed in Colab
2 !pip install -q transformers sentence-transformers
3 !pip install -q timm
4
5 # Core Imports
6 import os
7 import glob
8 import copy
9 import numpy as np
10 import pandas as pd
11 import cv2
12 import matplotlib.pyplot as plt
13 import matplotlib.gridspec as gridspec
14 import seaborn as sns
15 import urllib.request
16 import warnings
17 import logging
18 from collections import Counter
19 from PIL import Image as PILImage

```

```

20
21 # Suppress verbose output
22 warnings.filterwarnings('ignore')
23 logging.basicConfig(level=logging.CRITICAL)
24
25 # Scikit-learn
26 from sklearn.model_selection import (
27     train_test_split, StratifiedKFold, cross_val_score, GridSearchCV
28 )
29 from sklearn.preprocessing import StandardScaler
30 from sklearn.linear_model import LogisticRegression
31 from sklearn.svm import SVC
32 from sklearn.manifold import TSNE
33 from sklearn.decomposition import PCA
34 from sklearn.metrics import (
35     accuracy_score, confusion_matrix, classification_report,
36     ConfusionMatrixDisplay, roc_curve, auc, f1_score, precision_score,
37     recall_score
38 )
39
40 # PyTorch
41 import torch
42 import torch.nn as nn
43 import torch.optim as optim
44 from torch.utils.data import Dataset, DataLoader
45 import torchvision.transforms as transforms
46 import torchvision.models as tv_models
47
48 # HuggingFace (BLIP, ViT, MiniLM)
49 from transformers import (
50     BlipProcessor, BlipForConditionalGeneration,
51     ViTImageProcessor, ViTForImageClassification
52 )
53 from sentence_transformers import SentenceTransformer
54
55 # Colab: mount Google Drive
56 from google.colab import drive
57 drive.mount('/content/drive')
58
59 # Reproducibility
60 SEED = 42
61 np.random.seed(SEED)
62 torch.manual_seed(SEED)
63 if torch.cuda.is_available():
64     torch.cuda.manual_seed_all(SEED)
65
66 DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
67 print(f'Device: {DEVICE}')
68 print(f'PyTorch: {torch.__version__}')

```

A-1 Data Loading & Sample Grid

```

1 SHARED_DIR = '/content/drive/MyDrive/photos_for_ml/shared/'
2 PERSONAL_DIR = '/content/drive/MyDrive/photos_for_ml/personal/'
3
4 def load_image_paths(shared_dir, personal_dir):
5     """Scan directories for image files and assign binary labels.
6
7     Parameters:
8         shared_dir (str): path to directory containing shared (y=1) photos
9         personal_dir (str): path to directory containing personal (y=0) photos
10
11     Returns:
12         paths (list[str]): list of all image file paths
13         labels (list[int]): corresponding binary labels (1=shared, 0=personal)
14     """
15     extensions = ('*.jpg', '*.jpeg', '*.png', '*.JPG', '*.JPEG', '*.PNG')
16     shared_paths, personal_paths = [], []

```

```

17     for ext in extensions:
18         shared_paths += glob.glob(os.path.join(shared_dir, ext))
19         personal_paths += glob.glob(os.path.join(personal_dir, ext))
20     paths = shared_paths + personal_paths
21     labels = [1] * len(shared_paths) + [0] * len(personal_paths)
22     print(f'Shared (y=1): {len(shared_paths):>4}')
23     print(f'Personal (y=0): {len(personal_paths):>4}')
24     print(f'Total          : {len(paths):>4}')
25     return paths, labels
26
27 image_paths, labels = load_image_paths(SHARED_DIR, PERSONAL_DIR)
28 labels = np.array(labels)
29 # Sample image grid: 6 shared + 6 personal
30 def plot_sample_grid(image_paths, labels, n_per_class=6, seed=42):
31     """Display a 2x6 grid: top row shared, bottom row personal."""
32     rng = np.random.RandomState(seed)
33     paths_arr = np.array(image_paths)
34     labels_arr = np.array(labels)
35
36     fig, axes = plt.subplots(2, n_per_class, figsize=(n_per_class * 2.2, 5))
37     for row, (cls, name) in enumerate([(1, 'Shared'), (0, 'Personal')]):
38         cls_paths = paths_arr[labels_arr == cls]
39         picks = rng.choice(len(cls_paths), size=n_per_class, replace=False)
40         for col, idx in enumerate(picks):
41             img = cv2.imread(cls_paths[idx])
42             img = cv2.resize(img, (224, 224))
43             img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
44             axes[row, col].imshow(img)
45             axes[row, col].axis('off')
46             if col == 0:
47                 axes[row, col].set_ylabel(name, fontsize=11, fontweight='bold')
48     plt.suptitle('Sample Photos: Shared (top) vs Personal (bottom)', fontsize=12)
49     plt.tight_layout()
50     plt.savefig('fig_sample_grid.png', dpi=150, bbox_inches='tight')
51     plt.show()
52
53 plot_sample_grid(image_paths, labels, n_per_class=6)

```

A-2 Handcrafted Feature Extraction (Reused from Assignment 1)

```

1 # Haar cascade for face detection (Viola & Jones, 2001)
2 CASCADE_PATH = '/content/haarcascade_frontalface_default.xml'
3 if not os.path.exists(CASCADE_PATH):
4     url = ('https://raw.githubusercontent.com/opencv/opencv/master/'
5           'data/haarcascades/haarcascade_frontalface_default.xml')
6     urllib.request.urlretrieve(url, CASCADE_PATH)
7     print('Haar cascade downloaded.')
8 else:
9     print('Haar cascade already present.')
10 face_cascade = cv2.CascadeClassifier(CASCADE_PATH)
11
12 def extract_handcrafted_features(img_path, face_cascade, target_size=(256, 256)):
13     """Extract 13 handcrafted visual features from a single image.
14
15     Feature set (from A1): mean_brightness, brightness_var, mean_saturation,
16     saturation_var, warmth, colorfulness, mean_hue, hue_var, sharpness,
17     edge_density, center_weight, face_present, face_size_ratio.
18
19     Returns:
20     dict[str, float] or None if image cannot be loaded.
21     """
22     img_bgr = cv2.imread(img_path)
23     if img_bgr is None:
24         return None
25     img_bgr = cv2.resize(img_bgr, target_size)
26     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
27     img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV).astype(np.float32)

```

```

28 img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
29
30 H, S, V = img_hsv[:, :, 0], img_hsv[:, :, 1], img_hsv[:, :, 2]
31 R = img_rgb[:, :, 0].astype(float)
32 G = img_rgb[:, :, 1].astype(float)
33 B = img_rgb[:, :, 2].astype(float)
34
35 # Colour & lighting
36 mean_brightness = V.mean()
37 brightness_var = V.var()
38 mean_saturation = S.mean()
39 saturation_var = S.var()
40 warmth = R.mean() - B.mean()
41
42 # Colourfulness (Hasler & Susstrunk, 2003)
43 rg = R - G
44 yb = 0.5 * (R + G) - B
45 colorfulness = (np.sqrt(rg.std()**2 + yb.std()**2) +
46                 0.3 * np.sqrt(rg.mean()**2 + yb.mean()**2))
47
48 mean_hue = H.mean()
49 hue_var = H.var()
50
51 # Sharpness: Laplacian variance
52 sharpness = cv2.Laplacian(img_gray, cv2.CV_64F).var()
53
54 # Edge density: Canny
55 edges = cv2.Canny(img_bgr, threshold1=50, threshold2=150)
56 edge_density = edges.astype(bool).mean()
57
58 # Composition: centre brightness ratio
59 h, w = V.shape
60 mh, mw = h // 4, w // 4
61 center_crop = V[mh:h - mh, mw:w - mw]
62 center_weight = center_crop.mean() / (mean_brightness + 1e-6)
63
64 # Face detection
65 faces = face_cascade.detectMultiScale(
66     img_gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)
67 )
68 face_present = 1 if len(faces) > 0 else 0
69 face_size_ratio = (
70     max(wf * hf for (_, _, wf, hf) in faces) / (h * w) if face_present else 0.0
71 )
72
73 return dict(
74     mean_brightness=mean_brightness, brightness_var=brightness_var,
75     mean_saturation=mean_saturation, saturation_var=saturation_var,
76     warmth=warmth, colorfulness=colorfulness,
77     mean_hue=mean_hue, hue_var=hue_var,
78     sharpness=sharpness, edge_density=edge_density,
79     center_weight=center_weight, face_present=face_present,
80     face_size_ratio=face_size_ratio,
81 )
82
83 # Build DataFrame
84 print('Extracting handcrafted features...')
85 records = []
86 for path, label in zip(image_paths, labels):
87     feats = extract_handcrafted_features(path, face_cascade)
88     if feats is not None:
89         feats['label'] = int(label)
90         feats['filename'] = os.path.basename(path)
91         records.append(feats)
92
93 df = pd.DataFrame(records)
94 feature_cols = [c for c in df.columns if c not in ('label', 'filename')]
95 X_hand = df[feature_cols].values.astype(np.float64)
96 y = df['label'].values

```

```
97 print(f'Handcrafted feature matrix: {X_hand.shape}')
```

A-3 CNN Features — Frozen ResNet-18 (Reused from Assignment 2)

```
1 def build_cnn_feature_extractor(device):
2     """Load pretrained ResNet-18 with FC head removed.
3
4     Returns:
5         model: ResNet-18 in eval mode on device (outputs 512-d vectors)
6         preprocess: ImageNet-normalised 224x224 centre-crop pipeline
7     """
8     weights = tv_models.ResNet18_Weights.IMAGENET1K_V1
9     model = tv_models.resnet18(weights=weights)
10    model.fc = nn.Identity()
11    model = model.to(device)
12    model.eval()
13    preprocess = transforms.Compose([
14        transforms.Resize(256),
15        transforms.CenterCrop(224),
16        transforms.ToTensor(),
17        transforms.Normalize(mean=[0.485, 0.456, 0.406],
18                                std=[0.229, 0.224, 0.225]),
19    ])
20    return model, preprocess
21
22 def extract_cnn_features(image_paths, model, preprocess, device):
23     """Pass every image through the frozen CNN and collect 512-d features."""
24     features_list = []
25     failed = 0
26     for i, path in enumerate(image_paths):
27         try:
28             img = PILImage.open(path).convert('RGB')
29             img_tensor = preprocess(img).unsqueeze(0).to(device)
30             with torch.no_grad():
31                 feat = model(img_tensor)
32                 features_list.append(feat.cpu().numpy().flatten())
33         except Exception:
34             features_list.append(np.zeros(512))
35             failed += 1
36         if (i + 1) % 100 == 0:
37             print(f' {i+1}/{len(image_paths)} processed')
38     return np.array(features_list), failed
39
40 print('Loading ResNet-18...')
41 cnn_model, cnn_preprocess = build_cnn_feature_extractor(DEVICE)
42 print('Extracting CNN features...')
43 X_cnn, cnn_failed = extract_cnn_features(image_paths, cnn_model, cnn_preprocess, DEVICE)
44 print(f'CNN feature matrix: {X_cnn.shape} (failed: {cnn_failed})')
```

A-4 BLIP Caption Generation (NEW)

```
1 # Load BLIP captioning model
2
3 print('Loading BLIP...')
4 blip_processor = BlipProcessor.from_pretrained('Salesforce/blip-image-captioning-base')
5 blip_model = BlipForConditionalGeneration.from_pretrained(
6     'Salesforce/blip-image-captioning-base'
7 ).to(DEVICE)
8 blip_model.eval()
9
10 def generate_blip_caption(img_path, processor, model, device, max_length=50):
11     """Generate a single deterministic caption for one image using beam search."""
12     img = PILImage.open(img_path).convert('RGB')
13     inputs = processor(img, return_tensors='pt').to(device)
14     with torch.no_grad():
```

```

14         out = model.generate(**inputs, max_length=max_length, num_beams=3,
15                               repetition_penalty=1.8, no_repeat_ngram_size=2
16                               )
17         caption = processor.decode(out[0], skip_special_tokens=True)
18         return caption
19
20 # Generate captions for all 300 images
21 print('Generating BLIP captions (this takes a few minutes)...')
22 captions = []
23 for i, path in enumerate(image_paths):
24     try:
25         cap = generate_blip_caption(path, blip_processor, blip_model, DEVICE)
26     except Exception as e:
27         cap = ''
28         print(f' failed on {os.path.basename(path)}: {e}')
29     captions.append(cap)
30     if (i + 1) % 50 == 0:
31         print(f' {i+1}/{len(image_paths)} captioned')
32
33 df['caption'] = captions
34 print(f'\nGenerated {len(captions)} captions')
35 print('\nSample captions:')
36 for cls, name in [(1, 'Shared'), (0, 'Personal')]:
37     idxs = np.where(labels == cls)[0][:4]
38     print(f'\n--- {name} ---')
39     for idx in idxs:
40         print(f' "{captions[idx]}"')
41
42 # Save all captions to CSV for inspection
43 caption_df = pd.DataFrame({
44     'filename': [os.path.basename(p) for p in image_paths],
45     'true_label': labels,
46     'caption': captions
47 })
48 caption_df.to_csv('all_captions.csv', index=False)
49 print(f"Saved {len(caption_df)} captions to all_captions.csv")
50
51 from google.colab import files
52 files.download('all_captions.csv')
53
54 # Show a random sample of 20 shared and 20 personal captions
55 sample_shared = caption_df[caption_df['true_label'] == 1].sample(20, random_state=SEED)
56 sample_personal = caption_df[caption_df['true_label'] == 0].sample(20, random_state=SEED)
57 print("Sample Shared Captions:")
58 display(sample_shared[['filename', 'caption']])
59 print("\nSample Personal Captions:")
60 display(sample_personal[['filename', 'caption']])

```

A-5 Sentence Embedding — MiniLM (NEW)

```

1 print('Loading MiniLM...')
2 minilm_model = SentenceTransformer('all-MiniLM-L6-v2', device=DEVICE)
3
4 print('Encoding captions...')
5 X_caption = minilm_model.encode(captions, show_progress_bar=True,
6                                 convert_to_numpy=True)
7 print(f'Caption embedding matrix: {X_caption.shape}')

```

A-6 ViT Data Preparation (NEW)

```

1 # ViT uses the same ImageNet mean/std as the frozen ResNet pipeline
2 vit_train_transform = transforms.Compose([
3     transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
4     transforms.RandomHorizontalFlip(p=0.5),
5     transforms.ColorJitter(brightness=0.2, contrast=0.2,
6                             saturation=0.2, hue=0.1),

```

```

7     transforms.ToTensor(),
8     transforms.Normalize(mean=[0.485, 0.456, 0.406],
9                           std=[0.229, 0.224, 0.225]),
10 ])
11
12 vit_eval_transform = transforms.Compose([
13     transforms.Resize(256),
14     transforms.CenterCrop(224),
15     transforms.ToTensor(),
16     transforms.Normalize(mean=[0.485, 0.456, 0.406],
17                           std=[0.229, 0.224, 0.225]),
18 ])
19
20 class PhotoDataset(Dataset):
21     """Loads images from file paths and applies a given torchvision transform."""
22
23     def __init__(self, paths, labels, transform):
24         self.paths = list(paths)
25         self.labels = np.asarray(labels)
26         self.transform = transform
27
28     def __len__(self):
29         return len(self.paths)
30
31     def __getitem__(self, idx):
32         img = PILImage.open(self.paths[idx]).convert('RGB')
33         img = self.transform(img)
34         label = torch.tensor(int(self.labels[idx]), dtype=torch.long)
35         return img, label
36
37 print('ViT data pipeline ready.')

```

A-7 Exploratory Data Analysis

A-7.1 Cohen's d (Handcrafted Features)

```

1 def cohens_d(group0, group1):
2     """Pooled-variance Cohen's d between two 1-D arrays."""
3     n0, n1 = len(group0), len(group1)
4     var0, var1 = group0.var(ddof=1), group1.var(ddof=1)
5     pooled = np.sqrt(((n0 - 1) * var0 + (n1 - 1) * var1) / (n0 + n1 - 2))
6     return (group1.mean() - group0.mean()) / (pooled + 1e-9)
7
8 g0 = df[df['label'] == 0]
9 g1 = df[df['label'] == 1]
10 d_vals = {f: cohens_d(g0[f], g1[f]) for f in feature_cols}
11 d_series = pd.Series(d_vals).sort_values()
12
13 fig, ax = plt.subplots(figsize=(9, 5))
14 colors = ['#E07B54' if d > 0 else '#5B8DB8' for d in d_series.values]
15 ax.barh(d_series.index, d_series.values, color=colors)
16 ax.axvline(0, color='k', lw=0.8)
17 ax.axvline(0.5, color='gray', ls='--', lw=0.7, alpha=0.6)
18 ax.axvline(-0.5, color='gray', ls='--', lw=0.7, alpha=0.6)
19 ax.set_xlabel("Cohen's d (Shared Personal)")
20 ax.set_title('Handcrafted feature separability')
21 for i, (_, d) in enumerate(zip(d_series.index, d_series.values)):
22     x_text = d + 0.02 if d > 0 else d - 0.02
23     ha = 'left' if d > 0 else 'right'
24     ax.text(x_text, i, f'{d:+.2f}', va='center', ha=ha, fontsize=8)
25 plt.tight_layout()
26 plt.savefig('fig_cohens_d.png', dpi=150, bbox_inches='tight')
27 plt.show()
28
29 print('\nTop |d| features:')
30 print(d_series.reindex(d_series.abs().sort_values(ascending=False).index).head(5))

```

A-7.2 Caption Word Frequency

```
1 # Simple English stopwords list (no NLTK download required)
2 STOPWORDS = set("""
3 a an the and or but if then of to in on at by for with from as is are was were be been being
4 it its this that these those there here which who whom whose what where when why how
5 not no do does did have has had i you he she we they them his her their our your my me us
6 will would can could should may might up down out over into about very just also all
7 any some each one two s t
8 """).split()
9
10 def word_counts(caption_list, stopwords=STOPWORDS):
11     """Tokenise captions, lowercase, strip stopwords, return Counter."""
12     counter = Counter()
13     for cap in caption_list:
14         tokens = [w.lower().strip('.,!?:;"\'') for w in cap.split()]
15         tokens = [w for w in tokens if w and w not in stopwords and len(w) > 2]
16         counter.update(tokens)
17     return counter
18
19 shared_caps = [captions[i] for i in range(len(captions)) if labels[i] == 1]
20 personal_caps = [captions[i] for i in range(len(captions)) if labels[i] == 0]
21
22 shared_counts = word_counts(shared_caps)
23 personal_counts = word_counts(personal_caps)
24
25 top_shared = shared_counts.most_common(15)
26 top_personal = personal_counts.most_common(15)
27
28 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
29
30 words_s, counts_s = zip(*top_shared)
31 axes[0].barh(list(reversed(words_s)), list(reversed(counts_s)), color='#E07B54')
32 axes[0].set_title('Top 15 words in Shared captions')
33 axes[0].set_xlabel('Count')
34
35 words_p, counts_p = zip(*top_personal)
36 axes[1].barh(list(reversed(words_p)), list(reversed(counts_p)), color='#5B8DB8')
37 axes[1].set_title('Top 15 words in Personal captions')
38 axes[1].set_xlabel('Count')
39
40 plt.tight_layout()
41 plt.savefig('fig_word_freq.png', dpi=150, bbox_inches='tight')
42 plt.show()
43
44 # Print exclusive words
45 shared_only = set(dict(top_shared).keys()) - set(dict(top_personal).keys())
46 personal_only = set(dict(top_personal).keys()) - set(dict(top_shared).keys())
47 overlap = set(dict(top_shared).keys()) & set(dict(top_personal).keys())
48 print(f'\nShared-only (top 15): {sorted(shared_only)}')
49 print(f'Personal-only (top 15): {sorted(personal_only)}')
50 print(f'Overlap: {sorted(overlap)}')
```

A-7.3 Embedding Space: PCA and t-SNE

```
1 fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
2
3 # Panel (a): PCA on scaled handcrafted features
4 pca_hand = PCA(n_components=2, random_state=SEED)
5 X_hand_2d = pca_hand.fit_transform(StandardScaler().fit_transform(X_hand))
6 ve = pca_hand.explained_variance_ratio_
7 for cls, name, color, marker in [(0, 'Personal', '#5B8DB8', 'o'),
8                                   (1, 'Shared', '#E07B54', 's')]:
9     m = y == cls
10     axes[0].scatter(X_hand_2d[m, 0], X_hand_2d[m, 1], c=color, marker=marker,
11                    s=22, alpha=0.7, edgecolors='white', linewidths=0.3, label=name)
12 axes[0].set_xlabel(f'PC1 ({ve[0]*100:.1f}%)')
13 axes[0].set_ylabel(f'PC2 ({ve[1]*100:.1f}%)')
```

```

14 axes[0].set_title('Handcrafted (13d) PCA')
15 axes[0].legend(fontsize=8)
16 axes[0].grid(True, alpha=0.25)
17
18 # Panel (b): t-SNE on scaled caption embeddings
19 tsne_cap = TSNE(n_components=2, random_state=SEED, perplexity=30)
20 X_cap_2d = tsne_cap.fit_transform(StandardScaler().fit_transform(X_caption))
21 for cls, name, color, marker in [(0, 'Personal', '#5B8DB8', 'o'),
22                                  (1, 'Shared', '#E07B54', 's')]:
23     m = y == cls
24     axes[1].scatter(X_cap_2d[m, 0], X_cap_2d[m, 1], c=color, marker=marker,
25                    s=22, alpha=0.7, edgecolors='white', linewidths=0.3, label=name)
26 axes[1].set_xlabel('t-SNE 1')
27 axes[1].set_ylabel('t-SNE 2')
28 axes[1].set_title('Caption MiniLM (384d) t-SNE')
29 axes[1].legend(fontsize=8)
30 axes[1].grid(True, alpha=0.25)
31
32 plt.suptitle('Class separation across representations', fontsize=12, y=1.02)
33 plt.tight_layout()
34 plt.savefig('fig_embedding_comparison.png', dpi=150, bbox_inches='tight')
35 plt.show()

```

A-7.4 Class-Conditional Statistics

```

1 class_means = df.groupby('label')[feature_cols].mean().round(2)
2 class_means.index = ['Personal (y=0)', 'Shared (y=1)']
3 key = ['mean_brightness', 'brightness_var', 'face_present']
4 print('Class-conditional means (key features):')
5 print(class_means[key].T)

```

A-8 Train/Test Split & Scaling

```

1 # Stratified 80/20 split using a single index array
2 indices = np.arange(len(image_paths))
3 idx_train, idx_test = train_test_split(
4     indices, test_size=0.2, random_state=SEED, stratify=y
5 )
6
7 # Split every feature representation with the same indices
8 X_hand_train, X_hand_test = X_hand[idx_train], X_hand[idx_test]
9 X_cnn_train, X_cnn_test = X_cnn[idx_train], X_cnn[idx_test]
10 X_cap_train, X_cap_test = X_caption[idx_train], X_caption[idx_test]
11 y_train, y_test = y[idx_train], y[idx_test]
12
13 # Path lists for image-based models
14 train_paths = [image_paths[i] for i in idx_train]
15 test_paths = [image_paths[i] for i in idx_test]
16
17 # Scale each feature set (fit on train only)
18 scaler_hand = StandardScaler().fit(X_hand_train)
19 X_hand_train_s = scaler_hand.transform(X_hand_train)
20 X_hand_test_s = scaler_hand.transform(X_hand_test)
21
22 scaler_cnn = StandardScaler().fit(X_cnn_train)
23 X_cnn_train_s = scaler_cnn.transform(X_cnn_train)
24 X_cnn_test_s = scaler_cnn.transform(X_cnn_test)
25
26 scaler_cap = StandardScaler().fit(X_cap_train)
27 X_cap_train_s = scaler_cap.transform(X_cap_train)
28 X_cap_test_s = scaler_cap.transform(X_cap_test)
29
30 print('=' * 60)
31 print('DATA SPLIT')
32 print('=' * 60)
33 print(f'Train: {len(y_train):3d} (class 0: {np.sum(y_train==0)}, class 1: {np.sum(y_train==1)})')
34 print(f'Test : {len(y_test):3d} (class 0: {np.sum(y_test==0)}, class 1: {np.sum(y_test==1)})')

```

A-9 Logistic Regression on Handcrafted Features (Reused)

```
1 lr_hand = LogisticRegression(  
2     penalty='l2', C=1.0, solver='lbfgs', max_iter=1000, random_state=SEED  
3 )
```

A-10 SVM with RBF Kernel on CNN Features (Reused)

```
1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)  
2  
3 svm_param_grid = {  
4     'C': [0.01, 0.1, 1.0, 10.0, 100.0],  
5     'gamma': [0.001, 0.01, 0.1, 1.0, 'scale'],  
6 }  
7 svm_base = SVC(kernel='rbf', probability=True, random_state=SEED)  
8 svm_search = GridSearchCV(  
9     svm_base, svm_param_grid, cv=cv, scoring='accuracy', n_jobs=-1  
10 )
```

A-11 Caption LR: BLIP → MiniLM → Logistic Regression (NEW)

```
1 cap_lr_search = GridSearchCV(  
2     LogisticRegression(max_iter=2000, random_state=SEED,  
3         penalty='l2', solver='lbfgs'),  
4     {'C': [0.001, 0.01, 0.1, 1.0, 10.0]},  
5     cv=cv, scoring='accuracy', n_jobs=-1,  
6 )
```

A-12 Fine-Tuned ViT Setup (NEW)

```
1 def build_finetuned_vit(num_labels=2, device=DEVICE, n_unfrozen_blocks=4):  
2     """Load pretrained ViT-Base/16, replace classifier head, freeze all  
3     encoder blocks except the last 'n_unfrozen_blocks'.  
4     """  
5     model = ViTForImageClassification.from_pretrained(  
6         'google/vit-base-patch16-224',  
7         num_labels=num_labels,  
8         ignore_mismatched_sizes=True,  
9         attn_implementation="eager"  
10    )  
11  
12    model.config.output_attentions = True  
13  
14    # Freeze embeddings + all encoder layers initially  
15    for p in model.parameters():  
16        p.requires_grad = False  
17  
18    # Unfreeze last n_unfrozen_blocks transformer layers  
19    encoder_layers = model.vit.encoder.layer  
20    total_layers = len(encoder_layers)  
21    for layer in encoder_layers[total_layers - n_unfrozen_blocks:]:  
22        for p in layer.parameters():  
23            p.requires_grad = True  
24  
25    # Unfreeze final layernorm + classification head  
26    for p in model.vit.layernorm.parameters():  
27        p.requires_grad = True  
28    for p in model.classifier.parameters():  
29        p.requires_grad = True  
30  
31    model = model.to(device)  
32  
33    total = sum(p.numel() for p in model.parameters())  
34    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)  
35    print(f'Total parameters : {total:,}')  
36
```

```

36     print(f'Trainable parameters: {trainable:,} ({100*trainable/total:.1f}%)')
37     print(f'Frozen parameters    : {total - trainable:,}')
38     return model
39
40 print('Building ViT...')
41 vit_model = build_finetuned_vit(num_labels=2, device=DEVICE)

```

A-13 LR on Handcrafted — Cross-Validation

```

1 lr_hand.fit(X_hand_train_s, y_train)
2 cv_lr_hand = cross_val_score(lr_hand, X_hand_train_s, y_train,
3                               cv=cv, scoring='accuracy')
4 print(f'LR (Handcrafted): CV = {cv_lr_hand.mean():.3f}    {cv_lr_hand.std():.3f}')
5 print(f'  Fold scores: {np.round(cv_lr_hand, 3)}')

```

A-14 SVM RBF on CNN — Grid Search

```

1 print('SVM hyperparameter search (25 combos x 5 folds)...')
2 svm_search.fit(X_cnn_train_s, y_train)
3 best_svm = svm_search.best_estimator_
4 print(f'SVM (RBF, CNN): CV = {svm_search.best_score_:.3f}')
5 print(f'  Best C = {svm_search.best_params_["C"]}, '
6       f'best gamma = {svm_search.best_params_["gamma"]}')

```

A-15 Caption LR — Grid Search & Cross-Validation (NEW)

```

1 print('Caption LR hyperparameter search...')
2 cap_lr_search.fit(X_cap_train_s, y_train)
3 best_cap_lr = cap_lr_search.best_estimator_
4 best_C_cap = cap_lr_search.best_params_['C']
5 best_cv_cap = cap_lr_search.best_score_
6
7 # Fold-level scores at best C
8 best_idx = cap_lr_search.best_index_
9 cap_fold_scores = [cap_lr_search.cv_results_[f'split{i}_test_score'][best_idx]
10                    for i in range(5)]
11
12 print(f'Caption LR: CV = {best_cv_cap:.3f}    {np.std(cap_fold_scores):.3f}')
13 print(f'  Best C = {best_C_cap}')
14 print(f'  Fold scores: {np.round(cap_fold_scores, 3)}')
15
16 # Full grid results for reference
17 cap_cv_df = pd.DataFrame({
18     'C': [p['C'] for p in cap_lr_search.cv_results_['params']],
19     'CV Accuracy': cap_lr_search.cv_results_['mean_test_score'].round(3),
20     'CV Std': cap_lr_search.cv_results_['std_test_score'].round(3),
21 })
22 print('\nCaption LR grid:')
23 print(cap_cv_df.to_string(index=False))

```

A-16 ViT Fine-Tuning Loop with Early Stopping (NEW)

```

1 # ViT uses a single 80/20 split of the training set (same as A2 fine-tuning approach)
2 train_paths_arr = np.array(train_paths)
3
4 ft_train_idx, ft_val_idx = train_test_split(
5     np.arange(len(y_train)), test_size=0.2,
6     random_state=SEED, stratify=y_train
7 )
8
9 vit_train_ds = PhotoDataset(train_paths_arr[ft_train_idx],
10                             y_train[ft_train_idx], vit_train_transform)
11 vit_val_ds = PhotoDataset(train_paths_arr[ft_val_idx],

```

```

12         y_train[ft_val_idx], vit_eval_transform)
13
14 vit_train_loader = DataLoader(vit_train_ds, batch_size=16, shuffle=True,
15                               num_workers=2, pin_memory=True)
16 vit_val_loader = DataLoader(vit_val_ds, batch_size=16, shuffle=False,
17                             num_workers=2, pin_memory=True)
18
19 print(f'ViT train: {len(vit_train_ds)}, val: {len(vit_val_ds)}')
20
21 def train_vit(model, train_loader, val_loader, device,
22               epochs=25, lr=2e-5, weight_decay=1e-4, patience=5):
23     """Train ViT with AdamW + early stopping on validation loss.
24
25     AdamW (Loshchilov & Hutter, 2019) decouples weight decay from the adaptive
26     gradient step, which is the preferred regulariser for transformer fine-tuning.
27     """
28     criterion = nn.CrossEntropyLoss()
29     optimizer = optim.AdamW(
30         filter(lambda p: p.requires_grad, model.parameters()),
31         lr=lr, weight_decay=weight_decay
32     )
33
34     history = {'train_loss': [], 'val_loss': [],
35               'train_acc': [], 'val_acc': []}
36     best_val_loss = float('inf')
37     best_state = None
38     best_epoch = 0
39     patience_counter = 0
40
41     for epoch in range(epochs):
42         # Train
43
44         model.train()
45         running_loss, correct, total = 0.0, 0, 0
46         for imgs, lbls in train_loader:
47             imgs, lbls = imgs.to(device), lbls.to(device)
48             optimizer.zero_grad()
49             out = model(pixel_values=imgs).logits
50             loss = criterion(out, lbls)
51             loss.backward()
52             optimizer.step()
53             running_loss += loss.item() * imgs.size(0)
54             correct += (out.argmax(dim=1) == lbls).sum().item()
55             total += imgs.size(0)
56         train_loss = running_loss / total
57         train_acc = correct / total
58
59         # Validate
60
61         model.eval()
62         running_loss, correct, total = 0.0, 0, 0
63         with torch.no_grad():
64             for imgs, lbls in val_loader:
65                 imgs, lbls = imgs.to(device), lbls.to(device)
66                 out = model(pixel_values=imgs).logits
67                 loss = criterion(out, lbls)
68                 running_loss += loss.item() * imgs.size(0)
69                 correct += (out.argmax(dim=1) == lbls).sum().item()
70                 total += imgs.size(0)
71         val_loss = running_loss / total
72         val_acc = correct / total
73
74         history['train_loss'].append(train_loss)
75         history['val_loss'].append(val_loss)
76         history['train_acc'].append(train_acc)
77         history['val_acc'].append(val_acc)

```

```

77         # Early stopping
78
79         if val_loss < best_val_loss:
80             best_val_loss = val_loss
81             best_state = copy.deepcopy(model.state_dict())
82             best_epoch = epoch + 1
83             patience_counter = 0
84         else:
85             patience_counter += 1
86
87         print(f' Epoch {epoch+1:3d}: '
88               f'train_loss={train_loss:.4f} val_loss={val_loss:.4f} '
89               f'train_acc={train_acc:.3f} val_acc={val_acc:.3f}')
90
91         if patience_counter >= patience:
92             print(f' Early stopping at epoch {epoch+1} (best was epoch {best_epoch})')
93             break
94
95         # Restore best weights
96         if best_state is not None:
97             model.load_state_dict(best_state)
98         return model, history, best_epoch
99
100     print('\nTraining ViT...')
101     vit_model, vit_history, vit_best_epoch = train_vit(
102         vit_model, vit_train_loader, vit_val_loader, DEVICE,
103         epochs=25, lr=2e-5, weight_decay=1e-4, patience=5
104     )
105     print(f'\nBest epoch: {vit_best_epoch}')
106     print(f'Best val accuracy: {max(vit_history["val_acc"]):.3f}')

```

A-17 ViT Training Curves (NEW)

```

1 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
2 epochs_range = range(1, len(vit_history['train_loss']) + 1)
3
4 axes[0, 0].plot(epochs_range, vit_history['train_loss'], color='#E07B54', lw=1.8)
5 axes[0, 0].axvline(vit_best_epoch, color='gray', ls='--', lw=0.8,
6                    label=f'best epoch={vit_best_epoch}')
7 axes[0, 0].set_title('Train loss'); axes[0, 0].set_xlabel('Epoch')
8 axes[0, 0].grid(True, alpha=0.3); axes[0, 0].legend(fontsize=8)
9
10 axes[0, 1].plot(epochs_range, vit_history['val_loss'], color='#5B8DB8', lw=1.8)
11 axes[0, 1].axvline(vit_best_epoch, color='gray', ls='--', lw=0.8)
12 axes[0, 1].set_title('Val loss'); axes[0, 1].set_xlabel('Epoch')
13 axes[0, 1].grid(True, alpha=0.3)
14
15 axes[1, 0].plot(epochs_range, vit_history['train_acc'], color='#E07B54', lw=1.8)
16 axes[1, 0].axvline(vit_best_epoch, color='gray', ls='--', lw=0.8)
17 axes[1, 0].set_title('Train accuracy'); axes[1, 0].set_xlabel('Epoch')
18 axes[1, 0].grid(True, alpha=0.3)
19
20 axes[1, 1].plot(epochs_range, vit_history['val_acc'], color='#5B8DB8', lw=1.8)
21 axes[1, 1].axvline(vit_best_epoch, color='gray', ls='--', lw=0.8)
22 axes[1, 1].set_title('Val accuracy'); axes[1, 1].set_xlabel('Epoch')
23 axes[1, 1].grid(True, alpha=0.3)
24
25 plt.suptitle('ViT fine-tuning curves', fontsize=13, y=1.02)
26 plt.tight_layout()
27 plt.savefig('fig_vit_curves.png', dpi=150, bbox_inches='tight')
28 plt.show()
29
30 print(f'Train acc at best epoch: {vit_history["train_acc"][vit_best_epoch-1]:.3f}')
31 print(f'Val acc at best epoch : {vit_history["val_acc"][vit_best_epoch-1]:.3f}')
32 print(f'Train/val gap: '
33       f'{vit_history["train_acc"][vit_best_epoch-1] - vit_history["val_acc"][vit_best_epoch-1]:+.3f}')

```

A-18 All Model Predictions on Test Set

```

1 def evaluate_sklearn_model(model, X_test_s, y_test, name):
2     """Return predictions, probabilities, and metrics for an sklearn model."""
3     preds = model.predict(X_test_s)
4     if hasattr(model, 'predict_proba'):
5         probs = model.predict_proba(X_test_s)[: , 1]
6     else:
7         probs = model.decision_function(X_test_s)
8
9     acc = accuracy_score(y_test, preds)
10    f1 = f1_score(y_test, preds, average='macro')
11    fpr, tpr, _ = roc_curve(y_test, probs)
12    auc_val = auc(fpr, tpr)
13
14    return {
15        'name': name, 'preds': preds, 'probs': probs,
16        'accuracy': acc, 'f1': f1, 'auc': auc_val,
17        'fpr': fpr, 'tpr': tpr,
18    }
19
20 def evaluate_vit(model, test_paths, y_test, transform, device):
21     """Return predictions, softmax probabilities, metrics for the ViT."""
22     ds = PhotoDataset(test_paths, y_test, transform)
23     loader = DataLoader(ds, batch_size=16, shuffle=False)
24     model.eval()
25     all_preds, all_probs = [], []
26     with torch.no_grad():
27         for imgs, _ in loader:
28             imgs = imgs.to(device)
29             logits = model(pixel_values=imgs).logits
30             probs = torch.softmax(logits, dim=1)[: , 1]
31             preds = logits.argmax(dim=1)
32             all_preds.extend(preds.cpu().numpy())
33             all_probs.extend(probs.cpu().numpy())
34     preds = np.array(all_preds)
35     probs = np.array(all_probs)
36     acc = accuracy_score(y_test, preds)
37     f1 = f1_score(y_test, preds, average='macro')
38     fpr, tpr, _ = roc_curve(y_test, probs)
39     auc_val = auc(fpr, tpr)
40     return {
41         'name': 'Fine-tuned ViT', 'preds': preds, 'probs': probs,
42         'accuracy': acc, 'f1': f1, 'auc': auc_val,
43         'fpr': fpr, 'tpr': tpr,
44     }
45
46 # Evaluate each model on the 60-image test set
47 res_hand = evaluate_sklearn_model(lr_hand, X_hand_test_s, y_test, 'LR (Handcrafted)')
48 res_svm = evaluate_sklearn_model(best_svm, X_cnn_test_s, y_test, 'SVM RBF (CNN)')
49 res_cap = evaluate_sklearn_model(best_cap_lr, X_cap_test_s, y_test, 'Caption LR')
50 res_vit = evaluate_vit(vit_model, test_paths, y_test, vit_eval_transform, DEVICE)
51
52 all_results = {
53     'LR (Handcrafted)': res_hand,
54     'SVM RBF (CNN)': res_svm,
55     'Caption LR': res_cap,
56     'Fine-tuned ViT': res_vit,
57 }

```

A-19 Master Performance Table

```

1 rows = []
2 for name, r in all_results.items():
3     rows.append({
4         'Model': name,
5         'Test Accuracy': round(r['accuracy'], 3),
6         'F1 (Macro)': round(r['f1'], 3),

```

```

7         'AUC': round(r['auc'], 3),
8     })
9     perf_df = pd.DataFrame(rows)
10    print('=' * 70)
11    print('MASTER PERFORMANCE TABLE (Test Set, n=60)')
12    print('=' * 70)
13    print(perf_df.to_string(index=False))

```

A-20 ROC Curves

```

1 fig, ax = plt.subplots(figsize=(7, 7))
2 color_map = {
3     'LR (Handcrafted)': '#E07B54',
4     'SVM RBF (CNN)': '#2ECC71',
5     'Caption LR': '#8E44AD',
6     'Fine-tuned ViT': '#E74C3C',
7 }
8 for name, r in all_results.items():
9     ax.plot(r['fpr'], r['tpr'], color=color_map.get(name, 'gray'),
10            lw=2, label=f'{name} (AUC={r["auc"]:.3f})')
11 ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Random (AUC=0.500)')
12 ax.set_xlabel('False Positive Rate')
13 ax.set_ylabel('True Positive Rate')
14 ax.set_title('ROC curves: all models')
15 ax.legend(fontsize=9, loc='lower right')
16 ax.grid(True, alpha=0.25)
17 plt.tight_layout()
18 plt.savefig('fig_roc_all.png', dpi=150, bbox_inches='tight')
19 plt.show()

```

A-21 Confusion Matrices

```

1 fig, axes = plt.subplots(1, 4, figsize=(17, 4.2))
2 for ax, (name, r) in zip(axes, all_results.items()):
3     cm = confusion_matrix(y_test, r['preds'])
4     disp = ConfusionMatrixDisplay(cm, display_labels=['Personal', 'Shared'])
5     disp.plot(ax=ax, colorbar=False, cmap='Blues')
6     ax.set_title(f'{name} nAcc={r["accuracy"]:.3f}', fontsize=10)
7 plt.tight_layout()
8 plt.savefig('fig_cm_all.png', dpi=150, bbox_inches='tight')
9 plt.show()
10
11 # Print FP/FN breakdown
12 print('\nFP/FN breakdown:')
13 for name, r in all_results.items():
14     cm = confusion_matrix(y_test, r['preds'])
15     tn, fp, fn, tp = cm.ravel()
16     print(f' {name:20s} TN={tn:2d} FP={fp:2d} FN={fn:2d} TP={tp:2d}')

```

A-22 Caption Word Frequency & Prediction Examples (NEW)

```

1 # Caption examples: 3 correct shared, 3 correct personal, 1 FP, 1 FN
2 cap_preds = res_cap['preds']
3 cap_probs = res_cap['probs']
4 test_caps = [captions[i] for i in idx_test]
5
6 correct_shared = [i for i in range(len(y_test))
7                  if cap_preds[i] == 1 and y_test[i] == 1]
8 correct_personal = [i for i in range(len(y_test))
9                   if cap_preds[i] == 0 and y_test[i] == 0]
10 false_pos = [i for i in range(len(y_test))
11             if cap_preds[i] == 1 and y_test[i] == 0]
12 false_neg = [i for i in range(len(y_test))
13             if cap_preds[i] == 0 and y_test[i] == 1]
14

```

```

15 selected = (correct_shared[:3] + correct_personal[:3] +
16             false_pos[:1] + false_neg[:1])
17
18 case_rows = []
19 for local_idx in selected:
20     case_rows.append({
21         'filename': os.path.basename(test_paths[local_idx]),
22         'caption': test_caps[local_idx],
23         'true': 'Shared' if y_test[local_idx] == 1 else 'Personal',
24         'pred': 'Shared' if cap_preds[local_idx] == 1 else 'Personal',
25         'P(Shared)': round(float(cap_probs[local_idx]), 3),
26         'correct': '    ' if cap_preds[local_idx] == y_test[local_idx] else '    ',
27     })
28 case_df = pd.DataFrame(case_rows)
29 print('Caption examples:')
30 print(case_df.to_string(index=False))
31
32 # Top-5 and bottom-5 P(Shared) from training set
33 train_cap_probs = best_cap_lr.predict_proba(X_cap_train_s)[: , 1]
34 train_caps = [captions[i] for i in idx_train]
35
36 order = np.argsort(train_cap_probs)
37 bottom_5 = order[:5]
38 top_5 = order[-5:][::-1]
39
40 print('TOP 5 P(Shared) captions (training set):')
41 for i in top_5:
42     print(f'    {train_cap_probs[i]:.3f}    [true: {"Shared" if y_train[i]==1 else "Personal"}]    "{train_caps[i]}"')
43
44 print('\nBOTTOM 5 P(Shared) captions (training set):')
45 for i in bottom_5:
46     print(f'    {train_cap_probs[i]:.3f}    [true: {"Shared" if y_train[i]==1 else "Personal"}]    "{train_caps[i]}"')

```

A-23 Attention Rollout Implementation (NEW)

```

1 def compute_attention_rollout(model, img_tensor, device):
2     """Compute attention rollout map for a single ViT input.
3
4     Implementation:
5     1. Forward with output_attentions=True to get per-layer attention tensors
6       of shape (batch=1, n_heads, N+1, N+1).
7     2. Average across heads -> (L, N+1, N+1).
8     3. Add residual: A_hat = 0.5 * A_mean + 0.5 * I.
9     4. Multiply matrices across layers.
10    5. Take R[0, 1:] (CLS row, skip CLS self-entry), reshape to 14x14.
11
12    Returns:
13        spatial (np.ndarray): (14, 14) normalised heatmap in [0, 1]
14        predicted_class (int)
15    """
16    model.eval()
17    img_tensor = img_tensor.to(device)
18    with torch.no_grad():
19        outputs = model(pixel_values=img_tensor.unsqueeze(0),
20                        output_attentions=True)
21    # outputs.attentions is a tuple of length L, each (1, n_heads, N+1, N+1)
22    if outputs.attentions is None:
23        raise ValueError("Attentions not returned. Make sure model.config.output_attentions = True")
24
25    attentions = [a[0] for a in outputs.attentions]
26
27    # Mean across heads at each layer
28    avg_attentions = [a.mean(dim=0) for a in attentions] # list of (N+1, N+1)
29
30    n_tokens = avg_attentions[0].shape[0]
31    eye = torch.eye(n_tokens, device=device)

```

```

32
33 # Start with identity and multiply in layer order
34 rollout = torch.eye(n_tokens, device=device)
35 for A_mean in avg_attentions:
36     A_hat = 0.5 * A_mean + 0.5 * eye
37     # Row-normalise so each row sums to 1
38     A_hat = A_hat / A_hat.sum(dim=-1, keepdim=True)
39     rollout = A_hat @ rollout
40
41 # First row: attention flowing from CLS to all tokens
42 cls_attn = rollout[0, 1:] # skip CLS self-entry      length N=196
43 grid = int(np.sqrt(cls_attn.shape[0])) # 14
44 spatial = cls_attn.reshape(grid, grid).cpu().numpy()
45
46 # Normalise to [0, 1]
47 spatial = spatial - spatial.min()
48 if spatial.max() > 0:
49     spatial = spatial / spatial.max()
50
51 pred_class = int(outputs.logits.argmax(dim=1).item())
52 return spatial, pred_class
53
54 # Quick sanity check on one test image
55 sample_img = PILImage.open(test_paths[0]).convert('RGB')
56 sample_tensor = vit_eval_transform(sample_img)
57 sample_map, sample_pred = compute_attention_rollout(vit_model, sample_tensor, DEVICE)
58 print(f'Attention rollout shape: {sample_map.shape}, pred class: {sample_pred}')

```

A-24 Attention Map Visualisations (NEW)

```

1 # Pick: 2 correct shared, 2 correct personal, 1 FP, 1 FN (for the ViT)
2 vit_preds = res_vit['preds']
3 v_correct_shared = [i for i in range(len(y_test))
4                     if vit_preds[i] == 1 and y_test[i] == 1][:2]
5 v_correct_personal = [i for i in range(len(y_test))
6                      if vit_preds[i] == 0 and y_test[i] == 0][:2]
7 v_false_pos = [i for i in range(len(y_test))
8               if vit_preds[i] == 1 and y_test[i] == 0][:1]
9 v_false_neg = [i for i in range(len(y_test))
10              if vit_preds[i] == 0 and y_test[i] == 1][:1]
11
12 gallery = v_correct_shared + v_correct_personal + v_false_pos + v_false_neg
13 labels_gallery = (['Correct: Shared'] * len(v_correct_shared) +
14                  ['Correct: Personal'] * len(v_correct_personal) +
15                  ['False Positive'] * len(v_false_pos) +
16                  ['False Negative'] * len(v_false_neg))
17
18 n_examples = len(gallery)
19 fig, axes = plt.subplots(n_examples, 3, figsize=(12, 4 * n_examples))
20 if n_examples == 1:
21     axes = axes[np.newaxis, :]
22
23 for row, (local_idx, case_name) in enumerate(zip(gallery, labels_gallery)):
24     img = PILImage.open(test_paths[local_idx]).convert('RGB').resize((224, 224))
25     img_tensor = vit_eval_transform(PILImage.open(test_paths[local_idx]).convert('RGB'))
26     heatmap_14, _ = compute_attention_rollout(vit_model, img_tensor, DEVICE)
27     heatmap_224 = cv2.resize(heatmap_14, (224, 224))
28
29     true_str = 'Shared' if y_test[local_idx] == 1 else 'Personal'
30     pred_str = 'Shared' if vit_preds[local_idx] == 1 else 'Personal'
31     prob = res_vit['probs'][local_idx]
32
33     axes[row, 0].imshow(img)
34     axes[row, 0].set_title(
35         f'{case_name}\nTrue: {true_str} | Pred: {pred_str} | P(Shared)={prob:.3f}',
36         fontsize=9)
37     axes[row, 0].axis('off')
38

```

```

39 axes[row, 1].imshow(img)
40 axes[row, 1].imshow(heatmap_224, cmap='jet', alpha=0.5)
41 axes[row, 1].set_title('Attention overlay', fontsize=9)
42 axes[row, 1].axis('off')
43
44 axes[row, 2].imshow(heatmap_224, cmap='jet')
45 axes[row, 2].set_title('Rollout heatmap', fontsize=9)
46 axes[row, 2].axis('off')
47
48 plt.suptitle('ViT attention rollout examples gallery', fontsize=12, y=1.01)
49 plt.tight_layout()
50 plt.savefig('fig_attention_gallery.png', dpi=150, bbox_inches='tight')
51 plt.show()

```

A-25 Data Cut Analysis

```

1 # Reconstruct the test-set handcrafted DataFrame
2 df_test = df.iloc[idx_test].reset_index(drop=True)
3 brightness_median = df_test['mean_brightness'].median()
4
5 cuts = {
6     'Bright photos': df_test['mean_brightness'] >= brightness_median,
7     'Dark photos': df_test['mean_brightness'] < brightness_median,
8     'Face present': df_test['face_present'] == 1,
9     'No face': df_test['face_present'] == 0,
10 }
11
12 cut_rows = []
13 for cut_name, mask in cuts.items():
14     m = mask.values
15     n = int(m.sum())
16     if n < 3:
17         continue
18     for name, r in all_results.items():
19         acc = accuracy_score(y_test[m], r['preds'][m])
20         cut_rows.append({
21             'Data Cut': f'{cut_name} (n={n})',
22             'Model': name,
23             'Accuracy': round(acc, 3),
24         })
25
26 cut_df = pd.DataFrame(cut_rows)
27 cut_pivot = cut_df.pivot_table(index='Model', columns='Data Cut',
28                                 values='Accuracy', aggfunc='first')
29 print('Data cut table:')
30 print(cut_pivot.round(3).to_string())

```

A-26 All-Models-Wrong Cases

```

1 # For each test image, count how many of the 4 models got it wrong
2 wrong_counts = np.zeros(len(y_test), dtype=int)
3 for name, r in all_results.items():
4     wrong_counts += (r['preds'] != y_test).astype(int)
5
6 mostly_wrong_idx = np.where(wrong_counts >= 3)[0]
7 print(f'Images where >=3/4 models wrong: {len(mostly_wrong_idx)}')
8
9 all_wrong_idx = np.where(wrong_counts == 4)[0]
10 print(f'Images where ALL 4 models wrong: {len(all_wrong_idx)}')
11
12 # Show up to 6 of the mostly-wrong cases
13 show_idx = mostly_wrong_idx[:6] if len(mostly_wrong_idx) > 0 else []
14
15 if len(show_idx) > 0:
16     n_show = len(show_idx)
17     ncols = min(3, n_show)

```

```

18 nrows = int(np.ceil(n_show / ncols))
19 fig, axes = plt.subplots(nrows, ncols, figsize=(4 * ncols, 4 * nrows))
20 if nrows == 1 and ncols == 1:
21     axes = np.array([[axes]])
22 elif nrows == 1 or ncols == 1:
23     axes = axes.reshape(nrows, ncols)
24
25 rows_info = []
26 for k, local_idx in enumerate(show_idx):
27     r, c = k // ncols, k % ncols
28     img = PILImage.open(test_paths[local_idx]).convert('RGB').resize((224, 224))
29     axes[r, c].imshow(img)
30     axes[r, c].axis('off')
31     true_str = 'Shared' if y_test[local_idx] == 1 else 'Personal'
32     axes[r, c].set_title(f'True: {true_str}\nWrong: {wrong_counts[local_idx]}/4',
33                         fontsize=9)
34
35     rows_info.append({
36         'filename': os.path.basename(test_paths[local_idx]),
37         'true': true_str,
38         'caption': captions[idx_test[local_idx]],
39         'hand': 'S' if res_hand['preds'][local_idx] == 1 else 'P',
40         'svm': 'S' if res_svm['preds'][local_idx] == 1 else 'P',
41         'cap': 'S' if res_cap['preds'][local_idx] == 1 else 'P',
42         'vit': 'S' if res_vit['preds'][local_idx] == 1 else 'P',
43         'wrong': int(wrong_counts[local_idx]),
44     })
45
46 # Hide any unused panels
47 for k in range(n_show, nrows * ncols):
48     r, c = k // ncols, k % ncols
49     axes[r, c].axis('off')
50
51 plt.suptitle('Images where 3 /4 models are wrong', fontsize=12, y=1.01)
52 plt.tight_layout()
53 plt.savefig('fig_all_wrong.png', dpi=150, bbox_inches='tight')
54 plt.show()
55
56 print('\nPer-model predictions for these cases:')
57 print(pd.DataFrame(rows_info).to_string(index=False))
58 else:
59     print('No images with 3 model errors.')
60
61
62 print('=' * 70)
63 print('SUMMARY Assignment 3')
64 print('=' * 70)
65 print(f'LR (Handcrafted): CV={cv_lr_hand.mean():.3f} {cv_lr_hand.std():.3f}, Test={res_hand["accuracy"]:.3f}, AUC={res_hand["auc"]:.3f}')
66 print(f'SVM RBF (CNN): CV={svm_search.best_score:.3f}, Test={res_svm["accuracy"]:.3f}, AUC={res_svm["auc"]:.3f}')
67 print(f'Caption LR: CV={best_cv_cap:.3f}, Test={res_cap["accuracy"]:.3f}, AUC={res_cap["auc"]:.3f}')
68 print(f'Fine-tuned ViT: Best val={max(vit_history["val_acc"]:.3f) (epoch {vit_best_epoch}), Test={res_vit["accuracy"]:.3f}, AUC={res_vit["auc"]:.3f}')
69 print()
70 print(f'Caption LR best C: {best_C_cap}')
71 print(f'SVM best C, gamma: {svm_search.best_params_["C"]}, {svm_search.best_params_["gamma"]}')

```